also can we make this automation to combine with whatsapp. like the agentic ai bot will ask student login details through text. students rely with text message. it then askes to provide the excel sheet. then student reply with upload of excel sheet. then the ai will respond with mrng/evening. student reply and it simpy generates an ready made pdf. student can just download and check.

Yes, this is absolutely possible and will make the project stand out even more, because you'll have a **WhatsApp-first FFCS Agent** instead of just a web app.[1] [2]

How WhatsApp Flow Would Work

- Student saves your official number (WhatsApp Business Cloud / Twilio etc.) and sends "Hi".[3] [4]
- n8n WhatsApp Trigger workflow starts a **chat session** with memory.[5] [6]
- Bot asks:
  - VIT email ID (for basic verification; optional OTP in v2).
  - Asks the student to **attach the Excel** exported from FFCS (course list with slots, faculties, credits). n8n can download and read `.xlsx/.csv` from WhatsApp messages.[7] [8]
- Bot then asks "Morning, Evening, or Both?" and receives a simple text reply.[9]
- Backend + n8n:
  - Parse Excel → generate all valid timetables (using the FFCS rules and slot groups you described).
  - Use an n8n PDF generator template to render each timetable into a clean PDF (and optionally PNG images).[10] [11]
- Bot replies:
  - "Generated 5 best timetables for Morning batch – here's your PDF."
  - Sends the PDF as a **WhatsApp document** (file link or direct media).[12] [13]

Technical Pieces You'll Mention on Resume

- **WhatsApp Business Cloud API + n8n** for bi-directional chat and file handling.[14] [1]
- AI Agent in n8n that:
  - Orchestrates the conversation (login → file → preferences).
  - Calls your timetable generator service.
  - Returns final PDFs via WhatsApp.[15] [5]
- PDF generation pipeline inside n8n using a template workflow (e.g., HTML → PDFMonkey or n8n's free PDF flow).[11] [10]

Recommended Architecture Choice (Final)

- **Primary UX:** WhatsApp chatbot (students just chat and upload Excel).
- **Support UX:** Web dashboard (MERN) for those who want to upload via browser and visually inspect timetables.
- **Brain:** n8n workflows connecting:
  - WhatsApp ↔ Backend API ↔ PDF generator ↔ WhatsApp.[2] [9]

If you're okay including WhatsApp in v1, next step is "START DESIGN" and the focus can be:

- Data format for the Excel.
- API contract between n8n and your timetable service.
- Minimal WhatsApp conversation state machine (login → file → batch → deliver).

⁂

start design

Here is a concise **v1 design** that matches everything discussed (morning/evening batches, slot pairs, WhatsApp + web, n8n as brain).

1. Data Model (Excel → Backend)

1.1 Excel Format (one row per *faculty–course* option)

Columns (example):

- `courseCode` – e.g., CSE2001
- `courseName` – Data Structures and Algorithms
- `credits` – 3
- `facultyName` – Dr. XYZ
- `theorySlots` – `A1+TA1` or `C1+TC1+TCC1`
- `labSlots` – `L31+L32` or empty if no lab

- `room` – `AB-101`
- `batch` – `MORNING` or `EVENING` (optional; we can infer from theory slot series 1/2)[22] [23]

Backend will parse each row into JSON like:

```
{
  "courseCode": "CSE2001",
  "courseName": "Data Structures and Algorithms",
  "credits": 3,
  "facultyName": "Dr. XYZ",
  "theorySlots": ["A1", "TA1"],
  "labSlots": ["L31", "L32"],
  "room": "AB-101",
  "slotSeries": "1"       // derived from A1
}
```

All options for the same course become **different choices** (CSE2001 with different faculty/slots).

2. Backend Design (MERN, but focusing on server)

2.1 Core Collections

- **User**
  - `email` (VIT mail)
  - `name`
  - `role` (student/admin)
- **SemesterConfig**
  - `userId`
  - `semesterName`
  - `courses`: array of *CourseOption* objects (parsed from Excel)
  - `batchPreference` (MORNING/EVENING/BOTH)
- **GeneratedTimetable**
  - `userId`
  - `semesterConfigId`
  - `batch` (MORNING/EVENING)
  - `creditsTotal`
  - `courseOptions`: array of selected CourseOption IDs
  - `slotsUsed`: flat list of all primitive slots (e.g., `["A1","TA1","L31","L32", ...]`)

2.2 Slot & Clash Representation

- Maintain a **master slot map** in code (or DB) derived from FFCS grid (like the image / FFCS On The Go):[24] [25]

```
type SlotId = "A1" | "TA1" | "B1" | "L1" | "L2" | ...;

interface SlotMeta {
  day: "MON" | "TUE" | "WED" | "THU" | "FRI";
  start: string;   // "08:00"
  end: string;     // "08:50"
  isLab: boolean;
  series: 1 | 2;   // morning (1) or evening (2) for theory
}
```

- **CourseOption** uses *groups* but clash checking always expands groups to primitive slots.

2.3 Timetable Generation Algorithm (Core)

Input:

- `courseOptions[]` (from Excel, filtered by batch preference)
- `minCredits = 16`, `maxCredits = 27.5`[26] [27]

Steps (conceptual):

1. **Filter by batch:**
   - If `MORNING`:
     - Keep only options whose theorySlots are `*1` series (A1/B1/...) and labs in allowed lab range (e.g., L31–L60).
   - If `EVENING`:
     - Keep only `*2` series theory slots and labs in L1–L30.[28] [22]
2. **Group by courseCode:**
   - For each courseCode, you may have multiple faculty/slot options.
3. **Backtracking / DFS generator:**
   - Maintain:
     - `selectedOptions[]`
     - `usedSlots: Set<SlotId>`
     - `creditsTotal`
   - Iterate courses one by one; for each course:
     - Either skip that course (optional) OR pick exactly one of its options (for core courses you can mark as “must include” later).
     - When trying an option:

        - Expand theory + lab groups to `[primitive slots]`.
        - If any primitive slot already in `usedSlots` ⇒ clash ⇒ discard this branch.
        - If `creditsTotal + option.credits` > `maxCredits` ⇒ discard.
        - Else:
            - Add slots to `usedSlots`, add option to `selectedOptions`, update credits.
            - Recurse to next course.
    - At each stage where `creditsTotal` ∈ `[minCredits, maxCredits]`, add a copy of `selectedOptions` as a **valid timetable**.
4. **Post-processing:**
    - Remove duplicates (same slot set but different generation order).
    - Hard limit number of combinations (e.g., cap at 1000 per batch) to avoid explosion.

This logic aligns with constraint-programming approaches used in timetable generator papers. [29] [30]

3. n8n Design (Agentic Layer)

3.1 Main Workflows

1. **WhatsApp Conversation Workflow** [31] [32]
    - Trigger: WhatsApp Business Cloud node (message received).
    - States (store in DB or in n8n memory per user):
        - `ASK_EMAIL` → validate VIT mail format.
        - `ASK_EXCEL` → wait for media message; when `.xlsx/.csv` arrives, download. [33] [34]
        - Upload file to backend endpoint `/upload-excel` for that user.
        - `ASK_BATCH` → "Morning or Evening?"
        - On answer, call backend `/generate-timetables?batch=MORNING` via HTTP node.
        - Receive list of timetables (raw JSON).
2. **Ranking + Explanation Workflow** [35] [36]
    - HTTP Trigger from backend with:
        - User preferences (e.g., avoid 8 AM, max labs/day, prefer certain faculty).
        - List of valid timetables (credits, slots, courses).
    - **Agent 1 (Analyzer)**: Summarizes each timetable into features:
        - Early class count, gaps per day, number of labs, free days, etc.
    - **Agent 2 (Ranker)**: LLM node with prompt:
        - "You are a VIT FFCS assistant. Rank these timetables from best to worst based on: [preferences]. Return top 5 with scores 0–100 and brief reasons." [37] [38]
    - Return top-K ranked timetables to backend.

3. **PDF Generation + WhatsApp Reply**[39] [40]
    - For each selected timetable:
        - Use HTML → PDF node (or PDFMonkey integration) to render VIT-style timetable grid + course list.
    - Compress into:
        - One combined PDF for all top timetables, **or** one PDF per timetable.
    - WhatsApp node sends the PDF(s) back as documents to the user.[41] [31]

4. Web Frontend (Secondary but Resume-Relevant)

Simple React app (can be added after WhatsApp flow works):

- Pages:
    - **Login:** same email login as WhatsApp user.
    - **Upload:** same Excel file upload as chatbot.
    - **Preferences:** sliders + checkboxes.
    - **Results:** grid visualizer of each timetable; buttons:
        - "Download PDF", "Download PNG", "Share via WhatsApp".

Frontend largely calls the same backend endpoints used by n8n.

5. End-to-End Flow Summary

1. Student (Shree) → WhatsApp:
    - Sends "Hi".
    - Gives VIT email.
    - Uploads Excel.
    - Replies "Morning" or "Evening".
2. n8n:
    - Downloads Excel, passes to backend.
    - Calls `/generate-timetables` with batch + credit limits.
    - Sends valid timetables to ranking workflow.
    - Gets back top 5 ranked timetables + explanations.
    - Generates PDFs and sends via WhatsApp.
3. Backend:
    - Parses Excel → stores course options.
    - Runs clash-free timetable generator with slot groups and credit constraints.
    - Exposes JSON to n8n and optionally web UI.

If you want, next step can be:

- Detail the **Excel column spec + example row**, or
- Start with the **backend implementation plan** (file upload API + generator pseudocode).

⁂

do you remember how you guided me through the project 'lost-found-app' yesterday?

Yes. The same phased, copy-paste friendly style used for **ReclaimIt** can be repeated for this FFCS Agent project: clear phases, exact folder structure, and full file contents when you're ready to implement.[43] [44]

For this project, the phases can be:

- Phase 1: Backend scaffold (MERN project init + Excel upload API + basic slot model).
- Phase 2: Timetable generator (non-clashing algorithm with FFCS slot groups, morning/evening).
- Phase 3: n8n workflows (HTTP webhook API + ranking/explanation agents).
- Phase 4: WhatsApp flow (chat states, file upload, PDF send).
- Phase 5: Optional web UI dashboard.

Whenever you say **"Phase 1 start"**, the first response can be like we did earlier: concrete commands, folder structure, and first `.js` files ready to paste.

⁂

Phase 1 start

Phase 1 will mirror how the lost-found app started: small steps, backend first. For now, only **backend + Excel upload + basic parsing**, no n8n, no frontend.

1. Folder & Init (once)

In a new project folder, target structure:

- `ffcs-agent/`
  - `backend/`
    - `models/`
    - `routes/`
    - `utils/`
    - `uploads/` (Excel temp)
    - `server.js`

- `frontend/` (empty for now)

In terminal:

```
mkdir ffcs-agent
cd ffcs-agent
mkdir backend
cd backend
npm init -y
npm install express mongoose cors multer xlsx dotenv
npm install --save-dev nodemon
```

`xlsx` will parse Excel in Node using patterns similar to common guides.[46] [47]

Create `backend/package.json` scripts:

```
"scripts": {
  "dev": "nodemon server.js"
}
```

2. Basic server.js

Create `backend/server.js`:

```
const express = require('express');
const cors = require('cors');
const mongoose = require('mongoose');
const path = require('path');
require('dotenv').config();

const uploadRoutes = require('./routes/uploadRoutes');

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.use('/api/upload', uploadRoutes);

// Health check
app.get('/', (req, res) => {
  res.json({ status: 'FFCS Agent backend running' });
});

// Mongo connection (placeholder URI)
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/ffcs_agent';

mongoose
  .connect(MONGO_URI)
  .then(() => {
```

```
    console.log('✅ MongoDB connected');
    const PORT = process.env.PORT || 5000;
    app.listen(PORT, () => console.log(`🚀 Server running on port ${PORT}`));
  })
  .catch((err) => {
    console.error('❌ MongoDB connection error:', err.message);
    process.exit(1);
  });
```

This matches MERN best practices: single entry, CORS, JSON, routes folder.[48] [49]

Add `.env` (same folder):

```
MONGO_URI=mongodb://127.0.0.1:27017/ffcs_agent
```

3. Multer + Excel Upload Route

Create folder and file:

```
mkdir routes uploads
```

`backend/routes/uploadRoutes.js`:

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const fs = require('fs');
const xlsx = require('xlsx');

const router = express.Router();

// Multer storage: store Excel files in /uploads
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, path.join(__dirname, '..', 'uploads'));
  },
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    cb(null, `courses_${Date.now()}${ext}`);
  }
});

// Accept only .xls / .xlsx
const fileFilter = (req, file, cb) => {
  const allowed = ['.xls', '.xlsx'];
  const ext = path.extname(file.originalname).toLowerCase();
  if (!allowed.includes(ext)) {
    return cb(new Error('Only .xls or .xlsx files allowed'));
  }
  cb(null, true);
};
```

```
const upload = multer({ storage, fileFilter });

// POST /api/upload/courses
router.post('/courses', upload.single('file'), (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({ error: 'No file uploaded' });
    }

    const filePath = req.file.path;

    // Read Excel using xlsx
    const workbook = xlsx.readFile(filePath);
    const sheetName = workbook.SheetNames[^4_0];
    const sheet = workbook.Sheets[sheetName];

    // Convert to JSON (array of rows)
    const rows = xlsx.utils.sheet_to_json(sheet, { defval: '' });

    // Optional: delete the file after parsing
    fs.unlink(filePath, () => {});

    // For Phase 1: just echo parsed rows back to client
    return res.json({
      message: 'File parsed successfully',
      rowCount: rows.length,
      rows
    });
  } catch (err) {
    console.error('Upload parse error:', err);
    return res.status(500).json({ error: 'Failed to parse Excel file' });
  }
});

module.exports = router;
```

This follows typical Multer + XLSX patterns for Excel ingestion.[50] [46]

4. Define Expected Columns (Phase 1)

In Phase 1 you don't yet validate deeply, but **document** expected headings so you can create a sample Excel:

- `courseCode`
- `courseName`
- `credits`
- `facultyName`
- `theorySlots` – e.g. `A1+TA1`, `C1+TC1+TCC1`
- `labSlots` – e.g. `L31+L32` or empty

- `room`

`xlsx.utils.sheet_to_json` will use first row as keys, matching how most Node+Excel guides work.[51] [52]

5. How to Test Phase 1

1. Start Mongo locally (or use Atlas; URI already supports both).
2. In `backend` folder:

```
npm run dev
```

3. Use Postman or curl:

- Method: `POST`
- URL: `http://localhost:5000/api/upload/courses`
- Body: `form-data`
  - key: `file` (type: File)
  - value: your `.xlsx` with the columns above.

You should get JSON like:

```
{
  "message": "File parsed successfully",
  "rowCount": 12,
  "rows": [
    {
      "courseCode": "CSE2001",
      "courseName": "DSA",
      "credits": 3,
      "facultyName": "Dr. XYZ",
      "theorySlots": "A1+TA1",
      "labSlots": "L31+L32",
      "room": "AB-101"
    }
  ]
}
```

This confirms:

- Express server works.
- File upload pipeline works.
- Excel is parsed into structured JSON.

If you want, next step in Phase 1 can be:

- Add a `CourseOption` Mongoose model and save these rows to Mongo (with basic validation).

⁂

this is my current backend/package.json: {

"name": "backend",
 "version": "1.0.0",
 "description": "",
 "main": "index.js",
 "scripts": {
  "test": "echo "Error: no test specified" && exit 1"
 },
 "keywords": [],
 "author": "",
 "license": "ISC",
 "type": "commonjs",
 "dependencies": {
  "cors": "^2.8.5",
  "dotenv": "^17.2.3",
  "express": "^5.2.1",
  "mongoose": "^9.0.2",
  "multer": "^2.0.2",
  "xlsx": "^0.18.5"
 },
 "devDependencies": {
  "nodemon": "^3.1.11"
 }
}

Update your `backend/package.json` scripts so you can run `npm run dev`:

```
 {
   "name": "backend",
   "version": "1.0.0",
   "description": "",
   "main": "server.js",
   "scripts": {
     "dev": "nodemon server.js",
     "start": "node server.js",
     "test": "echo \"Error: no test specified\" && exit 1"
   },
   "keywords": [],
   "author": "",
   "license": "ISC",
   "type": "commonjs",
   "dependencies": {
     "cors": "^2.8.5",
     "dotenv": "^17.2.3",
     "express": "^5.2.1",
     "mongoose": "^9.0.2",
     "multer": "^2.0.2",
     "xlsx": "^0.18.5"
```

```
  },
  "devDependencies": {
    "nodemon": "^3.1.11"
  }
}
```

After this, keep the `server.js` and `routes/uploadRoutes.js` files exactly as in the previous message and run:

```
cd backend
npm run dev
```

Then test the Excel upload with Postman as described.

⁂

4. Define Expected Columns (Phase 1)

In Phase 1 you don't yet validate deeply, but document expected headings so you can create a sample Excel:
courseCode
courseName
credits
facultyName
theorySlots – e.g. A1+TA1, C1+TC1+TCC1
labSlots – e.g. L31+L32 or empty
room
xlsx.utils.sheet_to_json will use first row as keys, matching how most Node+Excel guides work.
what should i do in this step that you said

In this step you only need to prepare the **Excel template** that your backend will parse; no code change is required.[68] [69]

What to Do Now

1. **Create a sample Excel file**
    - Open Excel / Google Sheets.
    - In row 1, put these exact headers (one per column):
        - `courseCode`
        - `courseName`
        - `credits`
        - `facultyName`
        - `theorySlots`
        - `labSlots`

- `room`

2. **Fill 3–5 example rows**
   Example:

courseCode	courseName	credits	facultyName	theorySlots	labSlots	room
CSE2001	DSA	3	Dr. Alpha	A1+TA1	L31+L32	AB-101
MAT2002	Probability	4	Dr. Beta	B1+TB1		GDN-201
CSE3005	OS Lab	2	Dr. Gamma	C1+TC1+TCC1	L33+L34	SJT-310

   - Save as `.xlsx`, e.g., `sample_courses.xlsx`.

3. **Test with your API**
   - Start backend: `npm run dev`.
   - In Postman:
     - `POST http://localhost:5000/api/upload/courses`
     - Body → `form-data` → key `file` (type: File) → select `sample_courses.xlsx`.
   - If everything is correct, you should see the rows echoed as JSON with these keys, because `sheet_to_json` uses the first row as column names.[70] [71]

Once you confirm this works, the next step will be to **store these rows in Mongo with a CourseOption model** and start preparing for timetable generation.

⁂

Use Postman or curl:

Method: POST
URL: http://localhost:5000/api/upload/courses
Body: form-data
key: file (type: File)
value: your .xlsx with the columns above. how to do this

Here's exactly how to test the upload with Postman.

1. Make Sure Backend Is Running

In a terminal:

```
cd ffcs-agent/backend
npm run dev
```

You should see logs like "🚀 Server running on port 5000" and "✅ MongoDB connected".

2. Test in Postman (Step-by-step)

1. Open **Postman**.
2. Click **New** → **HTTP Request** (or use an existing tab).
3. **Set method and URL**
   - Method dropdown: select **POST**.
   - URL box: enter `http://localhost:5000/api/upload/courses`
4. **Select Body type**
   - Click the **Body** tab (just under the URL).
   - Select **form-data** (not raw / not x-www-form-urlencoded).
5. **Add the file field**
   - In the first row of the key/value table:
     - Under **Key**, type: `file`
     - On the same row, on the right of the key, there is a dropdown that usually says **Text** → click it and change to **File**.
     - In the **Value** column, click the "Select Files" button and choose your `sample_courses.xlsx`.

   It should look like:
   - Key: `file`
   - Type: `File`
   - Value: `sample_courses.xlsx` (or whatever your filename is).
6. **Send the request**
   - Click the **Send** button (top right).
7. **Check the response**
   - In the response area at the bottom, you should see JSON similar to:

```
{
  "message": "File parsed successfully",
  "rowCount": 3,
  "rows": [
    {
      "courseCode": "CSE2001",
      "courseName": "DSA",
      "credits": 3,
      "facultyName": "Dr. Alpha",
      "theorySlots": "A1+TA1",
      "labSlots": "L31+L32",
      "room": "AB-101"
    }
  ]
}
```

If you see an error instead, copy the **exact error response** (and any terminal error text) and paste it, then the next step can be fixing that.

⁂

how to open postman

To use Postman you first need it installed, then open it like any normal app.

1. Install Postman (if not installed)

On Windows (your case):

1. Open a browser and go to: https://www.postman.com/downloads/[74]
2. Download **Postman for Windows (64-bit)**.
3. Run the downloaded `.exe` and go through the installer (Next → Next → Finish).
4. After install, you may be asked to sign in or create a free account; you can skip sign-in if allowed.

2. Open Postman

After installation, you can open it in any of these ways:

- Press **Windows key**, type **"Postman"**, hit **Enter**.
- Or use **Start Menu** → **All apps** → **Postman**.
- Or double-click the **Postman** desktop shortcut if it was created.

Once Postman opens and you see the workspace, you can follow the previous steps to create the `POST http://localhost:5000/api/upload/courses` request and attach your Excel file.

⁂

i downloaded and logged in.

Perfect, you're on the Postman home screen.

Create the request from this screen

1. On the **left sidebar**, click the **orange pencil icon** (top-left) or look for a **"New"** or **"+ Request"** button.
2. Choose **"HTTP Request"** (or "New Request") if a menu appears.

Now you should see the main request UI with:

- A dropdown that says **GET**.
- A URL box next to it.
- Tabs like **Params / Auth / Headers / Body** below.

From there, follow these:

1. Change **GET** → **POST**.
2. In URL box, type: `http://localhost:5000/api/upload/courses`.
3. Click **Body** tab → choose **form-data**.
4. Add row:
   - Key: `file`
   - Type: change **Text** → **File** (small dropdown in that row).
   - Value: click “Select Files” and choose your `sample_courses.xlsx`.
5. Click **Send** (top-right).

Paste the JSON response or any error you see, and the next step can be storing this data into Mongo.

⁂

image.jpg

The 404 in your screenshot means the backend route `/api/upload/courses` does not exist (or server is not running) for that URL.

Check these quickly:

1. Confirm exact URL

Use **exactly**:

```
http://localhost:5000/api/upload/courses
```

No extra characters at the end. In your screenshot there seems to be an encoded `%0A` (newline) after `courses`. Delete everything after `courses` and type it again.

2. Confirm backend is running

In a terminal:

```
cd ffcs-agent/backend
npm run dev
```

You must see:

- ✅ `MongoDB connected` (or similar)
- 🚀 `Server running on port 5000`

If this is not printed, the API will 404.

3. Confirm route file is wired

In `backend/server.js` you should have:

```
const uploadRoutes = require('./routes/uploadRoutes');
app.use('/api/upload', uploadRoutes);
```

And `backend/routes/uploadRoutes.js` must contain:

```
router.post('/courses', upload.single('file'), (req, res) => {
  // ...
});
```

If any of these are missing or filename is different, the route won't be found and you'll get the same 404.

⁂

image.jpg

The new error is good progress: now the route works, but Multer cannot find the **uploads** folder.

Error (from screenshot):
`ENOENT: no such file or directory, open '...ffcs-agent\backend\uploads\courses_....xlsx'`

This means the directory `backend/uploads` does not exist.

Fix in 2 steps

1. Create the uploads folder

In terminal:

```
cd ffcs-agent/backend
mkdir uploads
```

Or in VS Code / Explorer, inside `backend` create a new folder named exactly:

```
uploads
```

No spaces, all lowercase.

2. Retry the Postman request

- Keep backend running (`npm run dev`).
- In Postman, **send** the same `POST http://localhost:5000/api/upload/courses` with your `file` field.

Now Multer will save the Excel into `backend/uploads`, `xlsx` will read it, and the API should respond with:

```
{
  "message": "File parsed successfully",
  "rowCount": ...,
  "rows": [ ... ]
}
```

Paste the JSON response here once you get it; then Phase 1 can move to saving these rows in Mongo.

⁂

ok?

Yes, this is perfect. Your Excel upload + parsing pipeline is working correctly (HTTP 200 with `message: "File parsed successfully"` and the rows listed).[85] [86]

Next logical step in Phase 1:

- Add a **CourseOption** Mongoose model.
- Save these parsed rows into Mongo when the file is uploaded.

When you're ready, say **"CourseOption model step"** and you'll get the exact `models/CourseOption.js` file and updated `uploadRoutes.js` to copy-paste.

⁂

CourseOption model step

Great, now Phase 1 will store those parsed rows in Mongo using a `CourseOption` model.

1. Create models folder and CourseOption model

If not already:

```
cd ffcs-agent/backend
mkdir models
```

Create `backend/models/CourseOption.js`:

```
const mongoose = require('mongoose');

const courseOptionSchema = new mongoose.Schema(
  {
    userEmail: {
      type: String,
      required: true
    },
    courseCode: {
      type: String,
      required: true,
      trim: true
    },
    courseName: {
      type: String,
      required: true,
      trim: true
    },
    credits: {
      type: Number,
      required: true,
      min: 0
    },
    facultyName: {
      type: String,
      required: true,
      trim: true
    },
    theorySlotsRaw: {
      type: String,
      required: true,
      trim: true
    },
    labSlotsRaw: {
      type: String,
      trim: true
    },
    room: {
      type: String,
      trim: true
    },
    // Derived fields for later logic
    theorySlots: {
      type: [String],
      default: []
    },
    labSlots: {
      type: [String],
      default: []
    },
    slotSeries: {
      type: Number, // 1 or 2 for morning/evening theory
      enum: [1, 2],
      required: false
    }
  },
```

```
  {
    timestamps: true
  }
);

module.exports = mongoose.model('CourseOption', courseOptionSchema);
```

This schema reflects typical MERN modelling with raw strings plus parsed arrays for later clash logic.[92] [93]

2. Update uploadRoutes to save parsed rows

Edit `backend/routes/uploadRoutes.js` to:

- Import the model.
- Accept a `userEmail` (for now we'll pass via query or body).
- Save each row as a `CourseOption`.

Replace the current file with this version:

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const fs = require('fs');
const xlsx = require('xlsx');
const CourseOption = require('../models/CourseOption');

const router = express.Router();

// Multer storage
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, path.join(__dirname, '..', 'uploads'));
  },
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    cb(null, `courses_${Date.now()}${ext}`);
  }
});

const fileFilter = (req, file, cb) => {
  const allowed = ['.xls', '.xlsx'];
  const ext = path.extname(file.originalname).toLowerCase();
  if (!allowed.includes(ext)) {
    return cb(new Error('Only .xls or .xlsx files allowed'));
  }
  cb(null, true);
};

const upload = multer({ storage, fileFilter });

// Helper: parse "A1+TA1" → ["A1","TA1"]
function splitSlots(raw) {
```

```
  if (!raw) return [];
  return String(raw)
    .split('+')
    .map((s) => s.trim())
    .filter(Boolean);
}

// POST /api/upload/courses?email=shree@vitstudent.ac.in
router.post('/courses', upload.single('file'), async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({ error: 'No file uploaded' });
    }

    const userEmail = req.query.email || req.body.email || 'unknown@vit.ac.in';

    const filePath = req.file.path;

    const workbook = xlsx.readFile(filePath);
    const sheetName = workbook.SheetNames[^13_0];
    const sheet = workbook.Sheets[sheetName];
    const rows = xlsx.utils.sheet_to_json(sheet, { defval: '' });

    fs.unlink(filePath, () => {});

    // Optional: clear previous options for this user for current run
    await CourseOption.deleteMany({ userEmail });

    const docsToInsert = rows.map((row) => {
      const theoryRaw = row.theorySlots || row.TheorySlots || '';
      const labRaw = row.labSlots || row.LabSlots || '';
      const theorySlots = splitSlots(theoryRaw);
      const labSlots = splitSlots(labRaw);

      // derive series from first theory slot: A1 → 1, A2 → 2
      let slotSeries = undefined;
      if (theorySlots.length > 0) {
        const lastChar = theorySlots[^13_0].slice(-1);
        const n = parseInt(lastChar, 10);
        if (n === 1 || n === 2) slotSeries = n;
      }

      return {
        userEmail,
        courseCode: row.courseCode || row.CourseCode || '',
        courseName: row.courseName || row.CourseName || '',
        credits: Number(row.credits || row.Credits || 0),
        facultyName: row.facultyName || row.FacultyName || '',
        theorySlotsRaw: theoryRaw,
        labSlotsRaw: labRaw,
        room: row.room || row.Room || '',
        theorySlots,
        labSlots,
        slotSeries
      };
    });
```

```
    const inserted = await CourseOption.insertMany(docsToInsert);

    return res.json({
      message: 'Courses stored successfully',
      rowCount: inserted.length,
      sample: inserted.slice(0, 3)
    });
  } catch (err) {
    console.error('Upload parse/save error:', err);
    return res.status(500).json({ error: 'Failed to parse or store Excel file' });
  }
});

module.exports = router;
```

This uses patterns similar to Excel–Mongo ingestion examples with Multer and `xlsx`.[94] [95]

3. Test saving to Mongo

1. Restart backend (for model import):

```
cd ffcs-agent/backend
npm run dev
```

2. In Postman, update URL to include your email:

```
POST http://localhost:5000/api/upload/courses?email=shree.vignesh@vitstudent.ac.in
```

- Keep body as `form-data`, key `file` (File), value `sample_courses.xlsx`.

3. Send the request.
   You should see:

```
{
  "message": "Courses stored successfully",
  "rowCount": 2,
  "sample": [
    {
      "_id": "...",
      "userEmail": "shree.vignesh@vitstudent.ac.in",
      "courseCode": "IMAT202L",
      "courseName": "Probability and Statistics",
      "credits": 3,
      "facultyName": "RAMESH KUMAR D",
      "theorySlotsRaw": "F2+TF2",
      "labSlotsRaw": "",
      "theorySlots": ["F2", "TF2"],
      "labSlots": [],
      "slotSeries": 2,
      ...
    }
```

```
  ]
}
```

If this works, Phase 1 is complete: Excel → API → Mongo with parsed slots. Next we can:

- Add an endpoint like `GET /api/courses?email=...` to verify data, then
- Start Phase 2: timetable generation (non-clashing combinations).

⁂

image.jpg

The error at the bottom says: **"Error: Invalid protocol: post http:"**. That means the URL bar in Postman starts with `post http://...` instead of `http://...`.

Fix the URL

In the URL box at the top:

1. Select the entire text.
2. Make sure it is exactly:

```
http://localhost:5000/api/upload/courses?email=shreevignesh.k2024@vitstudent.ac.in
```

- No `post` in front.
- No spaces or extra characters before `http`.

3. Confirm the method dropdown on the left is already **POST** (it is, from your screenshot).
4. Keep Body → form-data → `file` as before.
5. Click **Send** again.

If backend is running, you should now get JSON with `message: "Courses stored successfully"` and `rowCount`.

⁂

ok?

Yes, this is perfect. The response shows:

- `"message": "Courses stored successfully"`
- `rowCount: 2`
- `sample` array with your documents (userEmail, courseCode, theorySlots, slotSeries, etc.)

That confirms:

- Excel upload works.

- Rows are parsed and saved into Mongo via `CourseOption`.

Phase 1 backend ingest is complete.

Next backend step (still Phase 1, quick):

- Add a **GET** `/api/courses` endpoint to fetch all options for your email, so later n8n / frontend can reuse the same data.

If you want that now, say **"GET courses endpoint step"**.

⁂

GET courses endpoint step

Add a simple endpoint in the same `uploadRoutes.js` so you can fetch stored options per user.

1. Update `backend/routes/uploadRoutes.js`

Add this code **below** the POST route, before `module.exports = router;`:

```
// GET /api/upload/courses?email=...
router.get('/courses', async (req, res) => {
  try {
    const userEmail = req.query.email;
    if (!userEmail) {
      return res.status(400).json({ error: 'email query param is required' });
    }

    const courses = await CourseOption.find({ userEmail }).sort({
      courseCode: 1,
      facultyName: 1
    });

    return res.json({
      message: 'Courses fetched successfully',
      count: courses.length,
      courses
    });
  } catch (err) {
    console.error('Fetch courses error:', err);
    return res.status(500).json({ error: 'Failed to fetch courses' });
  }
});
```

No other changes needed in this file.

2. Test with Postman

1. Keep backend running: `npm run dev`.
2. New request in Postman:

- Method: **GET**
- URL:

```
http://localhost:5000/api/upload/courses?email=shreevignesh.k2024@vitstudent.ac.in
```

3. Click **Send**.

You should see:

```
{
  "message": "Courses fetched successfully",
  "count": 2,
  "courses": [
    {
      "_id": "...",
      "courseCode": "IMAT202L",
      "theorySlots": ["F2","TF2"],
      "labSlots": [],
      "slotSeries": 2,
      ...
    },
    ...
  ]
}
```

This finishes Phase 1: upload Excel → store in Mongo → read back by email.

⁂

const express = require('express');

const multer = require('multer');
const path = require('path');
const fs = require('fs');
const xlsx = require('xlsx');
const CourseOption = require('../models/CourseOption');

const router = express.Router();

// Multer storage
const storage = multer.diskStorage({
 destination: (req, file, cb) => {
  cb(null, path.join(__dirname, '..', 'uploads'));
 },
 filename: (req, file, cb) => {

```
    const ext = path.extname(file.originalname);
    cb(null, courses_${Date.now()}${ext});
  }
});

const fileFilter = (req, file, cb) => {
  const allowed = ['.xls', '.xlsx'];
  const ext = path.extname(file.originalname).toLowerCase();
  if (!allowed.includes(ext)) {
    return cb(new Error('Only .xls or .xlsx files allowed'));
  }
  cb(null, true);
};

const upload = multer({ storage, fileFilter });

// Helper: parse "A1+TA1" → ["A1","TA1"]
function splitSlots(raw) {
  if (!raw) return [];
  return String(raw)
    .split('+')
    .map((s) => s.trim())
    .filter(Boolean);
}

// POST /api/upload/courses?email=shree@vitstudent.ac.in
router.post('/courses', upload.single('file'), async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({ error: 'No file uploaded' });
    }

const userEmail = req.query.email || req.body.email || 'unknown@vit.ac.in';

const filePath = req.file.path;

const workbook = xlsx.readFile(filePath);
    const sheetName = workbook.SheetNames[0];
    const sheet = workbook.Sheets[sheetName];
    const rows = xlsx.utils.sheet_to_json(sheet, { defval: '' });

fs.unlink(filePath, () => {});

// Optional: clear previous options for this user for current run
    await CourseOption.deleteMany({ userEmail });

const docsToInsert = rows.map((row) => {
      const theoryRaw = row.theorySlots || row.TheorySlots || '';
      const labRaw = row.labSlots || row.LabSlots || '';
```

```
      const theorySlots = splitSlots(theoryRaw);
      const labSlots = splitSlots(labRaw);

// derive series from first theory slot: A1 → 1, A2 → 2
      let slotSeries = undefined;
      if (theorySlots.length > 0) {
        const lastChar = theorySlots[0].slice(-1);
        const n = parseInt(lastChar, 10);
        if (n === 1 || n === 2) slotSeries = n;
      }

return {
        userEmail,
        courseCode: row.courseCode || row.CourseCode || '',
        courseName: row.courseName || row.CourseName || '',
        credits: Number(row.credits || row.Credits || 0),
        facultyName: row.facultyName || row.FacultyName || '',
        theorySlotsRaw: theoryRaw,
        labSlotsRaw: labRaw,
        room: row.room || row.Room || '',
        theorySlots,
        labSlots,
        slotSeries
      };
    });

const inserted = await CourseOption.insertMany(docsToInsert);

return res.json({
      message: 'Courses stored successfully',
      rowCount: inserted.length,
      sample: inserted.slice(0, 3)
    });
  } catch (err) {
    console.error('Upload parse/save error:', err);
    return res.status(500).json({ error: 'Failed to parse or store Excel file' });
  }
});

module.exports = router; update this current file and return me entire code
```

Here is your updated `backend/routes/uploadRoutes.js` with both the POST (upload + save) and GET (fetch) endpoints:

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const fs = require('fs');
const xlsx = require('xlsx');
```

```
const CourseOption = require('../models/CourseOption');

const router = express.Router();

// Multer storage
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, path.join(__dirname, '..', 'uploads'));
  },
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    cb(null, `courses_${Date.now()}${ext}`);
  }
});

const fileFilter = (req, file, cb) => {
  const allowed = ['.xls', '.xlsx'];
  const ext = path.extname(file.originalname).toLowerCase();
  if (!allowed.includes(ext)) {
    return cb(new Error('Only .xls or .xlsx files allowed'));
  }
  cb(null, true);
};

const upload = multer({ storage, fileFilter });

// Helper: parse "A1+TA1" → ["A1","TA1"]
function splitSlots(raw) {
  if (!raw) return [];
  return String(raw)
    .split('+')
    .map((s) => s.trim())
    .filter(Boolean);
}

// POST /api/upload/courses?email=shree@vitstudent.ac.in
router.post('/courses', upload.single('file'), async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({ error: 'No file uploaded' });
    }

    const userEmail = req.query.email || req.body.email || 'unknown@vit.ac.in';

    const filePath = req.file.path;

    const workbook = xlsx.readFile(filePath);
    const sheetName = workbook.SheetNames[^17_0];
    const sheet = workbook.Sheets[sheetName];
    const rows = xlsx.utils.sheet_to_json(sheet, { defval: '' });

    fs.unlink(filePath, () => {});

    // Optional: clear previous options for this user for current run
    await CourseOption.deleteMany({ userEmail });
```

```
    const docsToInsert = rows.map((row) => {
      const theoryRaw = row.theorySlots || row.TheorySlots || '';
      const labRaw = row.labSlots || row.LabSlots || '';
      const theorySlots = splitSlots(theoryRaw);
      const labSlots = splitSlots(labRaw);

      // derive series from first theory slot: A1 → 1, A2 → 2
      let slotSeries = undefined;
      if (theorySlots.length > 0) {
        const lastChar = theorySlots[^17_0].slice(-1);
        const n = parseInt(lastChar, 10);
        if (n === 1 || n === 2) slotSeries = n;
      }

      return {
        userEmail,
        courseCode: row.courseCode || row.CourseCode || '',
        courseName: row.courseName || row.CourseName || '',
        credits: Number(row.credits || row.Credits || 0),
        facultyName: row.facultyName || row.FacultyName || '',
        theorySlotsRaw: theoryRaw,
        labSlotsRaw: labRaw,
        room: row.room || row.Room || '',
        theorySlots,
        labSlots,
        slotSeries
      };
    });

    const inserted = await CourseOption.insertMany(docsToInsert);

    return res.json({
      message: 'Courses stored successfully',
      rowCount: inserted.length,
      sample: inserted.slice(0, 3)
    });
  } catch (err) {
    console.error('Upload parse/save error:', err);
    return res.status(500).json({ error: 'Failed to parse or store Excel file' });
  }
});

// GET /api/upload/courses?email=...
router.get('/courses', async (req, res) => {
  try {
    const userEmail = req.query.email;
    if (!userEmail) {
      return res.status(400).json({ error: 'email query param is required' });
    }

    const courses = await CourseOption.find({ userEmail }).sort({
      courseCode: 1,
      facultyName: 1
    });

    return res.json({
```

```
      message: 'Courses fetched successfully',
      count: courses.length,
      courses
    });
  } catch (err) {
    console.error('Fetch courses error:', err);
    return res.status(500).json({ error: 'Failed to fetch courses' });
  }
});

module.exports = router;
```

Now:

- `POST http://localhost:5000/api/upload/courses?email=...` uploads + saves.
- `GET http://localhost:5000/api/upload/courses?email=...` returns all saved options for that email.

⁂

2. Test with Postman

Keep backend running: npm run dev.
New request in Postman:
Method: GET
URL: whats the body type

For the **GET** request you don't need any body type.

Use:

- Method: **GET**
- URL:

```
http://localhost:5000/api/upload/courses?email=shreevignesh.k2024@vitstudent.ac.in
```

Then:

- Leave **Body** tab on **none** (no form-data, no raw).
- Click **Send** and you should get the JSON with `courses`.

⁂

ok?

Yes, this is exactly right.

- `GET http://localhost:5000/api/upload/courses?email=shreevignesh.k2024@vitstudent.ac.in` is returning

- `"message": "Courses fetched successfully"`
- `count: 2`
- full `courses` array with parsed `theorySlots`, `labSlots`, `slotSeries`, etc.

That means Phase 1 backend ingestion is complete:

- Excel → upload
- Parse + save to Mongo
- Fetch by user email

Next major step (Phase 2) will be designing the **timetable generator** that takes these `CourseOption` docs and produces all non-clashing combinations for morning/evening with credits between 16 and 27.5.

When you want to start that, say: **"Phase 2 start"**.

⁂

Phase 2 start

Phase 2 will add the **timetable generator** that uses your saved `CourseOption` documents to produce all valid, non-clashing timetables for morning or evening within the credit range. [143] [144]

Below is design only (no code yet) so the logic is clear.

1. What the Generator Will Take In

From Mongo (using `CourseOption`):

- All course options for one user + one batch:
  - `courseCode`, `courseName`, `credits`
  - `facultyName`, `room`
  - `theorySlots` (array of primitive slots, e.g., `["F2","TF2"]`)
  - `labSlots` (array, e.g., `["L31","L32"]`)
  - `slotSeries` (1 or 2)

Input parameters:

- `batch`: `"MORNING"` or `"EVENING"`
- `minCredits = 16`, `maxCredits = 27.5`
- (Later) additional preferences (max labs/day, avoid 8 AM, etc.)

2. Preprocessing per Batch

When user chooses Morning or Evening:

- **Filter options by series and lab ranges**:
  - Morning:
    - `slotSeries === 1`
    - Lab slots in morning's allowed range (e.g., `L31–L60`).
  - Evening:
    - `slotSeries === 2`
    - Lab slots in `L1–L30`.[145] [146]
- **Group options by courseCode**:

```
{
  "IMAT202L": [ option1, option2, ... ],
  "CSE2001":  [ option3, option4, ... ],
  ...
}
```

Each course may have multiple faculty/slot combos; generator will choose **0 or 1** per course (0 if elective not taken).

3. Clash Checking Strategy

For clash detection:

- For a candidate timetable, maintain:
  - `usedSlots: Set<string>` – all primitive slots already taken (theory + lab).
  - `creditsTotal: number`.
- To test a new option:
  - Compute `allSlots = [...theorySlots, ...labSlots]`.
  - If **any** slot in `allSlots` is already in `usedSlots`, then the option clashes with chosen courses, because slot groups are atomic and cannot be broken.[147]
  - If `creditsTotal + option.credits > maxCredits`, reject branch.

Clash logic relies on your FFCS rule that groups like `A1+TA1`, `L1+L2` cannot be partially used.[146]

4. Backtracking Algorithm (Concept)

For N distinct courseCodes $C_1, C_2, \ldots, C_N$:

1. Define recursive function:
   - `dfs(index, selectedOptions, usedSlots, creditsTotal)`

- `index` = which courseCode we are deciding right now.

2. Base case:
   - If `index === N`:
     - If `creditsTotal` between 16 and 27.5, push `selectedOptions` as a valid timetable.
3. Recursive step for course `C_index`:
   - **Option A – Skip course** (especially electives):
     - Call `dfs(index + 1, selectedOptions, usedSlots, creditsTotal)`.
   - **Option B – Take one faculty/slot combo**:
     - Loop over `optionsForCourseCode`:
       - If this option clashes (any slot in `usedSlots`) → skip.
       - Else:
         - Create new `usedSlots2` = copy + allSlots for this option.
         - `credits2 = creditsTotal + option.credits`.
         - If `credits2 > maxCredits` → skip.
         - Add option to `selectedOptions2`.
         - Call `dfs(index + 1, selectedOptions2, usedSlots2, credits2)`.
4. While exploring, whenever `creditsTotal` is already ≥ `minCredits` and ≤ `maxCredits`, you can optionally record **intermediate** timetables even before processing all courses, but to keep logic simple at first, only accept at the end.

This is a typical backtracking/constraint-based approach used in academic timetable papers. [148] [149]

5. Output Structure

Each valid timetable will be stored/returned as:

```
{
  batch: "MORNING" | "EVENING",
  creditsTotal: number,
  courseOptions: [
    {
      courseCode,
      courseName,
      credits,
      facultyName,
      room,
      theorySlots,
      labSlots
    },
    ...
  ],
```

```
  slotsUsed: string[]   // all primitive slots, for quick visualization
}
```

Backend will:

- Save these in a `GeneratedTimetable` collection, or
- Return them directly as a JSON array to be ranked later by n8n.

6. Next Implementation Step

If this design matches what you expect, next step will be:

- Add a **new route** `POST /api/generate/timetables` that:
  - Reads courses for the user & batch from Mongo.
  - Runs the backtracking generator.
  - Returns an array of valid timetables (for now, unranked, with a cap like 200–500).

When you're ready for concrete code, say **"generate timetables route code"** and you'll get the full route file plus any helper needed, ready to paste.

⁂

generate timetables route code

Here is a complete new route file to generate timetables using the backtracking approach.

1. Create `backend/routes/generateRoutes.js`

```
const express = require('express');
const CourseOption = require('../models/CourseOption');

const router = express.Router();

// Credit limits
const MIN_CREDITS = 16;
const MAX_CREDITS = 27.5;

// Helper: check if any slot in newSlots is already used
function hasSlotClash(usedSlotsSet, newSlots) {
  for (const s of newSlots) {
    if (usedSlotsSet.has(s)) return true;
  }
  return false;
}

/**
 * Core generator using backtracking.
 * coursesByCode: Array of { courseCode, options: [CourseOption] }
 * batch: "MORNING" | "EVENING"
```

```
 */
function generateTimetables(coursesByCode, batch) {
  const results = [];
  const maxResults = 300; // safety cap

  function dfs(index, selectedOptions, usedSlotsSet, creditsTotal) {
    if (results.length >= maxResults) return;

    if (index === coursesByCode.length) {
      if (creditsTotal >= MIN_CREDITS && creditsTotal <= MAX_CREDITS) {
        const slotsUsed = Array.from(usedSlotsSet);
        results.push({
          batch,
          creditsTotal,
          courseOptions: selectedOptions.map((opt) => ({
            id: opt._id,
            courseCode: opt.courseCode,
            courseName: opt.courseName,
            credits: opt.credits,
            facultyName: opt.facultyName,
            room: opt.room,
            theorySlots: opt.theorySlots,
            labSlots: opt.labSlots
          })),
          slotsUsed
        });
      }
      return;
    }

    const { courseCode, options } = coursesByCode[index];

    // Option A: skip this course (common for electives)
    dfs(index + 1, selectedOptions, usedSlotsSet, creditsTotal);

    // Option B: include exactly one of the options
    for (const opt of options) {
      const allSlots = [...(opt.theorySlots || []), ...(opt.labSlots || [])];

      if (hasSlotClash(usedSlotsSet, allSlots)) continue;

      const credits2 = creditsTotal + (opt.credits || 0);
      if (credits2 > MAX_CREDITS) continue;

      const used2 = new Set(usedSlotsSet);
      for (const s of allSlots) used2.add(s);

      dfs(index + 1, [...selectedOptions, opt], used2, credits2);
    }
  }

  dfs(0, [], new Set(), 0);
  return results;
}

// POST /api/generate/timetables
```

```
// Body: { email: string, batch: "MORNING" | "EVENING" }
router.post('/timetables', async (req, res) => {
  try {
    const { email, batch } = req.body;

    if (!email || !batch) {
      return res.status(400).json({ error: 'email and batch are required' });
    }

    if (!['MORNING', 'EVENING'].includes(batch)) {
      return res.status(400).json({ error: 'batch must be MORNING or EVENING' });
    }

    // Fetch options for this user
    let options = await CourseOption.find({ userEmail: email });

    // Filter by batch / series
    if (batch === 'MORNING') {
      options = options.filter((opt) => opt.slotSeries === 1);
      // If you want to restrict labs to L31–L60 you can add a filter here later
    } else if (batch === 'EVENING') {
      options = options.filter((opt) => opt.slotSeries === 2);
      // Similarly, restrict labs to L1–L30 later if needed
    }

    if (options.length === 0) {
      return res.status(400).json({ error: 'No course options found for this user/batch'
    }

    // Group by courseCode
    const map = new Map();
    for (const opt of options) {
      if (!map.has(opt.courseCode)) {
        map.set(opt.courseCode, []);
      }
      map.get(opt.courseCode).push(opt);
    }

    const coursesByCode = Array.from(map.entries()).map(([courseCode, opts]) => ({
      courseCode,
      options: opts
    }));

    const timetables = generateTimetables(coursesByCode, batch);

    return res.json({
      message: 'Timetables generated',
      count: timetables.length,
      timetables
    });
  } catch (err) {
    console.error('Generate timetables error:', err);
    return res.status(500).json({ error: 'Failed to generate timetables' });
  }
});
```

```
module.exports = router;
```

This structure follows typical constraint/backtracking timetable designs in research, but simplified to FFCS slot groups and credit constraints.[174] [175]

2. Wire route in `backend/server.js`

At the top, add:

```
const generateRoutes = require('./routes/generateRoutes');
```

And after the upload route:

```
app.use('/api/generate', generateRoutes);
```

3. Test in Postman

1. Ensure you already uploaded the Excel for your email (Phase 1).
2. New request:

- Method: **POST**
- URL:

```
http://localhost:5000/api/generate/timetables
```

- Body → `raw` → JSON:

```
{
  "email": "shreevignesh.k2024@vitstudent.ac.in",
  "batch": "EVENING"
}
```

- Headers: `Content-Type: application/json`.

3. Send.

You should get something like:

```
{
  "message": "Timetables generated",
  "count": 3,
  "timetables": [
    {
      "batch": "EVENING",
      "creditsTotal": 7,
      "courseOptions": [ ... ],
```

```
      "slotsUsed": ["F2","TF2","L1","L2",...]
    }
  ]
}
```

If the count is 0, it just means with your tiny sample and 16–27.5 limits there is no combination; for testing you can temporarily change `MIN_CREDITS` to 0 or 3 in `generateRoutes.js` and retry.

⁂

image.jpg

The 200 OK with `"count": 0` means the algorithm ran correctly but, with your **current data and credit limits**, it couldn't find any timetable between 16 and 27.5 credits.

Right now your sample Excel only has **2 courses** (3 + 1 credits = 4), which can never reach the minimum 16 credits, so every combination is rejected by the `MIN_CREDITS` check.[184]

You have two options for testing:

Option A: Temporarily lower credit limits (for testing)

In `backend/routes/generateRoutes.js`:

```
const MIN_CREDITS = 0;
const MAX_CREDITS = 30;
```

Save, restart `npm run dev`, and send the same POST again.
Now you should see `count` ≥ 1 with timetables containing your 1–2 sample courses.

Option B: Use a fuller Excel

Create a bigger sample Excel with many courses (total possible credits > 27.5), then upload again and keep:

```
const MIN_CREDITS = 16;
const MAX_CREDITS = 27.5;
```

Either way, the behavior (0 results) is expected for just two low-credit rows; the code path itself is working correctly.

⁂

if we know its working mean lets proceed with next

Good. With ingestion + generator working, next high-value step is to **prepare for n8n + WhatsApp and UI** by exposing a clean API and slightly improving the generator output.

Given the length so far, here is a minimal “next” that keeps momentum:

1. Add simple “preview” endpoint (for frontend/n8n)

Create `backend/routes/previewRoutes.js`:

```
const express = require('express');
const CourseOption = require('../models/CourseOption');

const router = express.Router();

// GET /api/preview/summary?email=...
router.get('/summary', async (req, res) => {
  try {
    const email = req.query.email;
    if (!email) return res.status(400).json({ error: 'email is required' });

    const courses = await CourseOption.find({ userEmail: email });

    const totalCourses = courses.length;
    const totalCredits = courses.reduce((sum, c) => sum + (c.credits || 0), 0);

    const bySeries = {
      series1: courses.filter((c) => c.slotSeries === 1).length,
      series2: courses.filter((c) => c.slotSeries === 2).length
    };

    return res.json({
      message: 'Summary ready',
      email,
      totalCourses,
      totalCredits,
      bySeries
    });
  } catch (err) {
    console.error('Preview summary error:', err);
    return res.status(500).json({ error: 'Failed to build summary' });
  }
});

module.exports = router;
```

Wire in `server.js`:

```
const previewRoutes = require('./routes/previewRoutes');
```

```
app.use('/api/preview', previewRoutes);
```

This gives n8n/WhatsApp a **lightweight check** ("your file has 18 courses, 54 credits, 10 in series1, 8 in series2") before running heavy generation.

2. Choose Where to Go Next

Pick one:

- **A. n8n/WhatsApp side**: design the webhook contract n8n will hit:
  - `/api/upload/courses`
  - `/api/generate/timetables`
  - `/api/preview/summary`
    Then plan WhatsApp conversation states.
- **B. Web frontend**: minimal React page that:
  - Lets you upload Excel.
  - Shows summary + a sample of generated timetables.

Reply with **"Path A (n8n)"** or **"Path B (frontend)"**, and the next steps will focus there.

⁂

can you decide yourself?

Path A: **n8n + WhatsApp** first. This matches your agentic AI goal and makes the project feel unique faster than a basic web UI.[203] [204]

For next steps (when you're ready to continue, not now):

1. Define the **HTTP contract** that n8n will use:
   - Step 1: `/api/upload/courses` (already done).
   - Step 2: `/api/preview/summary` (optional quick check).
   - Step 3: `/api/generate/timetables` (already done).
2. In n8n:
   - Create a **WhatsApp Business Cloud** inbound workflow:
     - Node 1: WhatsApp trigger → capture user's number and text.[205] [206]
     - Node 2: Conversation logic:
       - Ask for VIT email.
       - Ask for Excel (media message) and forward the file via HTTP node to `/api/upload/courses?email=...`.[207] [208]
       - Ask for batch ("Morning" / "Evening") and call `/api/generate/timetables`.
       - Send back top 3 timetables as raw JSON for now.

3. Only after this, add:
    - PDF generation node in n8n.[209]

When you want to continue, say **“n8n WhatsApp flow step 1”** and the exact nodes + payload examples to configure inside n8n will be outlined.

⁂

n8n WhatsApp flow step 1

Step 1 is to set up a **WhatsApp trigger workflow in n8n** and make sure you can receive simple messages from your number. This is just plumbing; no Excel or AI yet.[219] [220]

1. Prerequisites (one-time)

- A WhatsApp Business Cloud app + credentials (App ID, Client Secret, Phone Number ID, Permanent Token). n8n docs show the exact fields.[221] [222]
- Your n8n instance reachable from the internet (n8n Cloud or self-hosted with a public URL / tunnel like ngrok).

If you haven’t set up WhatsApp Business Cloud yet, that would be a separate step; assuming you at least know where credentials are.

2. Create the Workflow in n8n

1. Open n8n → click **New Workflow**.
2. Click the big **“+”** button on the canvas.
3. Search for **“WhatsApp Business Cloud”**.
4. Under operations, choose **Trigger** → **Message** (or “On messages”, depending on version). This is the WhatsApp Trigger node.[223] [220]
5. Rename the node to: `WhatsApp Trigger`.
6. In the node’s **Credentials** section:
    - Click **Add New** → select **WhatsApp Business Cloud**.
    - Fill the fields (App ID, Client Secret, etc.) according to the credential docs.[221]
    - Save credentials.
7. In the trigger node:
    - Event type: **Messages**.
    - Verify URL / webhook registration will be handled automatically when you click “Execute” (n8n docs explain; often you must paste n8n webhook URL into Meta’s webhook config).[224]

For now, leave other options default.

3. Add a Simple Reply Node

1. Add a second node: **WhatsApp Business Cloud** (action node, not trigger).
2. Operation: **Send** → **Message**.[225] [226]
3. Credentials: same as trigger.
4. "To" field: use the expression from the trigger message, e.g.:
   - Click the **gear** → **Add Expression**, then pick something like:
     `{{$json["messages"][^25_0]["from"]}}`

   (Exact path depends on the schema; you can inspect the trigger output once you receive a test message, then adjust.)
5. Message body: hard-code for now:

   ```
   Hi Shree , FFCS Agent here. I received your message!
   ```

6. Connect **WhatsApp Trigger** → **Send Message**.

4. Test the Flow

1. Click **Execute Workflow** (or **Test** / **Listen for test event** depending on n8n version) to start listening.
2. From your phone, send any WhatsApp message to your WhatsApp Business number.
3. In n8n:
   - You should see a new execution with the trigger node having your message payload.
   - The second node should send the reply; you see the message come back on your phone.

If this works, the WhatsApp + n8n connection is verified.

If you confirm this step works (simple "echo" reply), next step will be:

- Design the **conversation states** (ask email → ask for Excel → download media → call your `/api/upload/courses` endpoint).

⁂

i dont have prerequisites so first lets finish that and proceed next things.ok?

Yes, that's the right order. For now, park n8n/WhatsApp until you have:

- A WhatsApp Business Cloud app created in Meta
- Phone number connected
- Permanent access token
- Webhook URL registered (will use n8n URL later)[248] [249]

Because setting those up involves your personal Facebook/Meta account and UI clicks, it's better you follow an official step-by-step guide:

- Meta WhatsApp Cloud API + n8n:[250] [248]
- Example walkthroughs:[251] [252]

Meanwhile, your backend is already good enough to showcase:

- Excel upload + parse
- Store in Mongo
- Generate FFCS timetables for a batch

Next productive thing (while you or a friend handle Meta setup) is to build a **simple React frontend** that talks to these APIs (upload Excel, see summary, see generated timetables). If you want that now, say **"Frontend Phase start"** and it can be scaffolded like we did for ReclaimIt.

⁂

you need to guide me to complete the prerequisites. not wait for me

Understood. Let's set up the **WhatsApp Business Cloud** prerequisites step-by-step so later you can plug it into n8n.

1. Create/Use a Facebook Developer Account

1. Go to https://developers.facebook.com and log in with your Facebook account.[262]
2. Accept any developer terms if prompted.

2. Create a Meta App for WhatsApp

1. In the top-right, click **My Apps** → **Create App**.
2. Choose **Business** or **Other** (WhatsApp supports both; "Business" is recommended).[262]
3. App name: e.g., `FFCS-Agent-App`.
4. Add your email, click **Create App**.

3. Add WhatsApp Product and Test Number

1. Inside the app dashboard, in the left sidebar, find **Add Product**.
2. Under **WhatsApp**, click **Set up**.[262]
3. On the **Getting Started** page:
    - Meta will give you a **temporary test phone number** (you can keep it for dev) and
    - A **temporary access token**.

4. Enter your own WhatsApp number as a **recipient** and click **Send message** from their demo box to confirm it works.

This confirms your Meta WhatsApp Cloud API itself is alive.[263] [262]

4. Generate a Permanent Access Token

The default token expires quickly; n8n needs a long-lived token.

1. In your app, go to **Business Settings** → **Accounts** → **WhatsApp Accounts** (or follow the "System User" token instructions from the WhatsApp Business Cloud docs).[262]
2. Create a **System User** if not already present.
3. Assign it the WhatsApp permissions (e.g., `whatsapp_business_messaging`, `whatsapp_business_management`).
4. Generate a **Permanent Access Token** for this system user.
5. Copy and save it securely; you'll paste this into n8n credentials later.

(Exact UI labels can move slightly; follow the "WhatsApp Business Cloud credentials" doc alongside: it lists each click path.)[262]

5. Get the Phone Number ID and WhatsApp Business Account ID

Still in the WhatsApp section of your app:[264] [262]

- You'll see:
  - **Phone Number ID** (for the test number or any business number you connect).
  - **WhatsApp Business Account ID**.

Copy both; these are required in n8n's WhatsApp credentials form.

6. Prepare Webhook for n8n (but don't wire yet)

Later, when n8n is running with a public URL:

1. In the Meta app sidebar, go to **WhatsApp** → **Configuration** → **Webhook**.[265] [263]
2. You'll add:
   - **Callback URL**: your n8n WhatsApp Trigger URL.
   - **Verify Token**: any secret string you choose (you will also put this inside the n8n WhatsApp Trigger node).

For now, just note that you know where this setting is; you'll return once n8n is up.

7. Summary of What You Must Have Ready

By the end of prerequisites you should have:

- **App ID** (from Meta app).
- **App Secret** (from Settings → Basic).
- **WhatsApp Business Account ID**.
- **Phone Number ID** (for the test/real number).
- **Permanent Access Token** with WhatsApp permissions.
- Knowledge of where to set **Webhook URL + Verify Token**.

All these go into the **WhatsApp Business Cloud** credentials in n8n.[266] [264]

Once you complete these steps in Meta (you can tell when you see messages working from the Meta "Try it" console), come back and say **"n8n WhatsApp credentials step"** and the exact fields to fill in the n8n UI will be given, followed by the trigger + reply workflow.

⁂

image.jpg

The Meta screen is saying **"Sorry, a temporary error has occurred. Please try again in a little while."** when you enter your mobile number. That is an issue on Meta's side, not your setup.

Try these practical fixes:

1. Basic retrial steps

- Wait 5–10 minutes and click **Send Verification SMS** again.
- Refresh the page (Ctrl+R) and re-enter the number.
- Try logging out of Facebook, log back in, then open https://developers.facebook.com and resume registration.[276]

2. Check number / format

- Confirm **Country** is India (+91).
- Enter your number **without leading 0** (e.g., `9871411556`, not `09871411556`).
- Ensure this number is not already used for another disabled or blocked developer account; Meta sometimes rejects numbers tied to policy issues.[277]

3. Try alternative verification

On the same screen you can see a note: "You can also verify your account by adding a credit card."

- Click the **"adding a credit card"** link.

- If you have a debit/credit card and are comfortable, use that method instead of SMS; Meta treats card verification as equivalent.[277]

4. If it persists

If after multiple attempts (and maybe trying from another browser or network) you get the same "temporary error", this is a Meta platform problem; you may need to:

- Try again later in the day.
- Or contact Meta support / Help Center with a screenshot.[276]

Until Meta lets you finish **"Verify your account"**, they won't allow creating the WhatsApp app. Your backend and timetable generator are already working; once this verification step succeeds, the flow to add WhatsApp and connect to n8n can continue from "Create a Meta App for WhatsApp".

⁂

image.jpg

Great, you're on the **"My Apps"** page and ready to create the WhatsApp app.

Do this:

1. Create the Meta App

1. Click the green **"Create App"** button in the middle.
2. A dialog appears asking for the app type:
   - Choose **"Business"** (or **"Other"** if Business is not available, but Business is recommended for WhatsApp).[288]
3. Click **Next**.
4. Fill details:
   - **App name**: `FFCS-Agent-App` (or any name).
   - **App contact email**: your email.
   - Leave Business Account as default (None) for now if you don't have one.
5. Click **Create app** and complete any captcha.

You should land on the **App Dashboard** for your new app.

2. Add the WhatsApp Product

Once the app is created:

1. In the left sidebar, scroll to **"Add product"**.
2. Find **WhatsApp** and click **"Set up"**.[289] [288]
3. You'll be taken to the **WhatsApp > Getting Started** page with:

- Temporary test phone number
- Temporary access token
- WhatsApp Business Account ID
- Phone Number ID

Stop there for a moment; once you reach that screen, send a screenshot or say “on WhatsApp Getting Started page”, and the next instructions will be:

- How to send a test message to yourself from Meta’s console.
- How to generate a long-lived token.

⁂

image.jpg

On this “Use cases” screen, choose the WhatsApp option so Meta sets up the app correctly.

What to select now

1. In the list, scroll (you already see it) to:
   “Connect with customers through WhatsApp”
   Description: “Start a WhatsApp conversation, send notifications, create ads that click-to-WhatsApp…” [301]
2. Click the **radio button** next to “Connect with customers through WhatsApp” so it is selected.
3. Click the **Next** button at the bottom/right.
4. On the following steps:
   - **Business**: if it asks about a business, you can choose your personal business account or “No Business Account” / create later (for development it still works).
   - **Requirements**: keep defaults, continue.
   - **Overview**: confirm and click **Create app**.

Once done, you should land on the app dashboard with the WhatsApp product available. Then:

- In the left sidebar click **WhatsApp → Getting Started**.

When you reach that “Getting Started” page showing a test phone number and token, send “on Getting Started page” and the next steps (test message + generate long-lived token) will be given.

⁂

image.jpg

This dialog is asking you to create a **Business Portfolio**, which is fine—even if you're just a student, you can treat it as a one-person "business" for dev purposes.[314]

Fill it like this:

- **Business portfolio name**:
  - `FFCS Agent Labs` or `Shree Tech Projects` (any simple name, no special characters).
- **First name**: your first name (e.g., `Shree`).
- **Last name**: your last name.
- **Business email**: your main email (e.g., VIT or personal Gmail).

Then click **Create portfolio**.

Meta will attach this portfolio to your app so you can use WhatsApp Business Cloud. After that:

1. You should go back to the app creation flow.
2. Continue until the app is created and the **WhatsApp "Getting Started"** page appears in the left sidebar.

Once you're on the **WhatsApp Getting Started** page (with test number and token shown), say "on Getting Started page" and the next instructions will walk through test message + token + IDs to note down.

⁂

i created now wh next

Now connect the **WhatsApp** product inside this app.

1. Go to WhatsApp "Getting Started"

1. You should be on the app dashboard now.
2. In the left sidebar, look for **"Add product"** or a section called **"WhatsApp"**.
3. Click **"Set up"** next to **WhatsApp**.[328] [329]
4. You'll be taken to **WhatsApp → Getting Started**.

On that page you should see:

- A **temporary test phone number** (provided by Meta).
- A **Temporary access token**.
- **WhatsApp Business Account ID** and **Phone Number ID**.

2. Send a test message

On the same Getting Started page:

1. There's a section "Send and receive messages" with:
    - A field to enter your own WhatsApp number.
    - A "Template message" preview.[328]
2. Enter your WhatsApp number (with country code, e.g., `+91...`).
3. Click **"Send message"**.

You should receive a WhatsApp message from the Meta test number on your phone. This confirms the Cloud API works.

After you reach this point and see the test message on your phone, reply here with **"test message received"**, and the next step will be:

- Generate a **long-lived access token** and
- Note **Phone Number ID** + **Business Account ID** for n8n.

⁂

wait i m stuck here

You're on the **app dashboard**, which is good; now you just need to open the WhatsApp section.

Follow these clicks:

1. In the **left sidebar**, click **"Use cases"** (it's just below "Required actions").
2. On the Use cases page, you should see **"Connect with customers through WhatsApp"** listed (since you chose that earlier).
3. Click that WhatsApp use case; there should be a button or link like **"Continue setup"** or **"Get started"**.[343]

If you don't see WhatsApp in "Use cases", then:

- Scroll the left sidebar down to the **"App settings"** or **"Products"** section and look for **"WhatsApp"** directly.[344]
- Click **WhatsApp** → it should take you to the **"Getting Started"** page with the test number and token.

Once you reach a page whose title is **"Getting Started"** and shows a WhatsApp test phone number and a "Send message" box, tell me "on Getting Started page" and we'll do the test message + token step.

⁂

image.jpg

Perfect, this is the **WhatsApp Business Platform – Quickstart** screen.

Do this now

1. Ensure the **business portfolio** dropdown shows `FFCS Agent Labs` (it already does).
2. Tick any required checkboxes (you have one about agreeing to terms, already implied here).
3. Click the blue **"Continue"** button at the bottom right.

After clicking Continue, Meta will:

- Provision a **test WhatsApp phone number** for you.
- Take you to a page where you can send a **test message** to your own WhatsApp number and see:
  - WhatsApp Business Account ID
  - Phone Number ID
  - Temporary access token

Once you reach that "Send and receive messages" / "Step 1: Send a test message" page, say **"on test message page"** or send a screenshot, and the next instructions will be: how to send the test message and which IDs/tokens to copy for n8n.

⁂

image.jpg

Now you're on the **WhatsApp Business Platform – Quickstart** page with a big "Test the API" button.

Meta hides the detailed IDs & phone info behind that. Do this:

1. Open the Test panel

1. Click the blue **"Test the API"** button.
2. A right-side panel should open (or a new section below) titled something like **"Send and receive messages"** or **"Test your app"**.[374]

In that panel you should see:

- A **From** test WhatsApp number (owned by Meta).
- A field to enter your **To** number (your personal WhatsApp).
- A **Template message** preview and a **"Send message"** button.
- Somewhere in the same quickstart you'll also see:
  - **Temporary access token**

- **WhatsApp Business Account ID**
- **Phone Number ID**

2. Send a test message

1. Enter your own WhatsApp number (with +91).
2. Click **Send message**.
3. Check your phone: you should get a message from Meta's test number.

When you've received the message, reply here **"test message received"** and then the next step will be:

- Note exactly which values to copy (Business Account ID, Phone Number ID, temporary token) for n8n, and
- Later generate a long-lived token.

⁂

image.jpg

Perfect—this is exactly the **API Testing** / "Send test messages" page.

1. Note the IDs

From section **2. Select a 'From' phone number**:

- **WhatsApp Business Account ID**: `847330254844371` (from your screenshot).
- **Phone number ID**: `839904665883864`.

Copy these somewhere safe; you'll paste them into n8n later.[391] [392]

2. Generate a temporary access token and send a test

1. In section **1. Generate a temporary access token**:
   - Click **"Generate access token"**.
   - Log in/confirm if it asks.
   - Click **Copy** to copy the token string (starts with `EAAG...`). Save it for quick tests (valid 60 minutes).[391]
2. In section **3. Add a recipient phone number**:
   - Open the dropdown **"Select a recipient number"**.
   - Choose **"Add phone number"**.
   - Enter your WhatsApp number with country code (e.g., `+91...`).
   - Confirm and send the OTP if asked.
3. Scroll to section **6. Send messages with the API** and click the **"Send message"** button (or use section 5's template send).

    - Using this UI is easier than running curl.
4. Check your phone: you should receive a WhatsApp message from the **test number +1 555 169 2793**.

Once you see that message, reply here **“test message received”** and then next step will be:

- Plan n8n **credentials** using:
    - WhatsApp Business Account ID
    - Phone Number ID
    - Permanent token (we’ll convert from temporary to long-lived later)

⁂

““test message received”” **access token: EAALZAIY1tHfUBQZAbimZCbsKjVuUd9vNuAtpZAnZCtFK5MPmHgQJuHOSROBdPKhZCXaZCqkoD6mrE830TMFN0rSkn9eWYfYW1SJiqy087ZCnlifzZAA4vJXmuHufHchDqkR5L4O6YvH9HAUor5jCaN1hyvNhp3ZBJKAhlcUKkO8LUsGU6gZCDEqHxPZAEtlPkLcVAl16tZBiKEsypZAGd7vHyFEjP5hZBbgautikCVoW5a3xJKT9wntH8EtvslOqhuZCkgiNRnAMJtZBnPJtPlYTJsUlezZCnutFEZC and i received my message from that number.**

Great—that confirms your WhatsApp Cloud API is working end-to-end.[410]

Now treat this token as **secret** (never commit it to Git/GitHub); you will paste it only into n8n’s credentials.

What You Have Ready Now

- WhatsApp Business Account ID: `847330254844371`.
- Phone Number ID: `839904665883864`.
- Temporary access token (works for ~60 minutes).[410]

This is enough to:

- Create **WhatsApp Business Cloud credentials** in n8n.
- Build the simple trigger + reply workflow described earlier.[411] [412]

When you’re ready to continue inside n8n, say **“n8n WhatsApp credentials step”** and the exact fields to fill in the n8n UI (and how to test the first echo bot) will be given.

⁂

n8n WhatsApp credentials step

Now you'll create **WhatsApp Business Cloud credentials in n8n** using the IDs and token you just got.[430] [431]

1. Open Credentials in n8n

1. Open your n8n instance in the browser.
2. Left sidebar → click **Credentials** (key icon).
3. Click **"Add credential"** (top-right).
4. Search for **"WhatsApp Business Cloud"** and select it.[430]

2. Fill WhatsApp Business Cloud Credential Fields

In the credential form, fill:

- **Name**: `WhatsApp FFCS Agent` (any label).
- **Access Token**: paste your temporary token (`EAALZAlY1tHfU...`).
- **Phone Number ID**: `839904665883864`.
- **WhatsApp Business Account ID**: `847330254844371`.

Leave other advanced options default for now.

Click **Save**.

n8n will use this token + IDs to send/receive messages through the test number.[432] [430]

3. Next Step After Credentials

Once saved, the next step is to:

- Create a new Workflow
- Add **WhatsApp Trigger** node using this credential
- Add **WhatsApp Send Message** node to echo back

When you're ready for that, say **"n8n echo workflow step"** and you'll get exact node settings + test flow.

⁂

only this two are there

That's fine—your n8n build is showing the **generic "WhatsApp API"** credential, which is what you should use for the WhatsApp Cloud API.[450] [451]

Choose:

- **"WhatsApp API"** (not OAuth).

Then fill fields as follows (names may differ slightly but map like this):

- **Name**: `WhatsApp FFCS Agent`.
- **Access Token / API Key / Bearer Token**: paste your token `EAALZAlY1tHfU...`.
- **Phone Number ID**: `839904665883864`.
- **WhatsApp Business Account ID**: `847330254844371`.

If the form also has:

- **Base URL**: leave default (`https://graph.facebook.com/v20.0` or similar).[450]

Save the credential.

After saving, next step is to build the echo workflow:

If you want that now, say **"n8n echo workflow step"** and you'll get:

- WhatsApp Trigger node config using this credential.
- WhatsApp Send node config to reply "Hi Shree, FFCS Agent here" when you send any message.

⁂

image.jpg

This credential looks correct for the fields shown:

- Access Token: your EAAL... token
- Business Account ID: `847330254844371`
- Allowed HTTP Request Domains: `All`

Two notes:

1. If there is a **Details** tab, you can give it a friendly name like `WhatsApp FFCS Agent`, but that is optional.
2. Some n8n versions require **Phone Number ID** to be configured in each WhatsApp node instead of in the credential; that's fine—we'll set it in the node.[470]

Click **Save**.

When saved successfully, you're ready for the **echo workflow**. If you want to proceed, say **"n8n echo workflow step"** and the settings for the trigger node + send node will be given so

you can test chatting from your phone through n8n.

⁂

n8n echo workflow step

Now you'll build a minimal **WhatsApp echo bot** in n8n using this credential.

1. Create New Workflow

1. In n8n, click **Workflows** → **New**.
2. Name it: `FFCS WhatsApp Echo`.

2. Add WhatsApp Trigger Node

1. Click the **+** on canvas → search **"WhatsApp"** → choose **WhatsApp Trigger** (or similar "Webhook trigger for WhatsApp API"; the icon matches your credential).[490]
2. In the node:

- **Credentials**: select `WhatsApp account` (the one you created).
- **Phone Number ID**: if there is a field, paste `839904665883864`.
- **Resource / Operation**: choose **"Messages" / "On incoming message"** or similar (the default for trigger).

3. Click **Save** on the node.

(If the node shows a **Webhook URL** and **Verify Token**, we'll use that later for Meta webhooks; for pure polling/testing some builds work without it.)

3. Add WhatsApp Send Node (Echo)

1. Add second node: **WhatsApp API** (action node).
2. Connect: drag from **WhatsApp Trigger** → **WhatsApp API**.
3. Configure the send node:

- **Credentials**: same `WhatsApp account`.
- **Operation / Resource**: **Send Message**.
- **Phone Number ID**: `839904665883864` (if required here).

For the **To** field:

- Click the value field → **Add Expression**.
- Pick from the trigger data the sender's number; typical path for Cloud API triggers is something like:

```
{{$json["entry"][^41_0]["changes"][^41_0]["value"]["messages"][^41_0]["from"]}}
```

If your trigger shows a different structure, you will adjust later; for now use this and we'll fix after first run.

For the **Message body**:

- Set to:

```
Hi Shree, FFCS Agent here. I received: "{{$json["entry"][^41_0]["changes"][^41_0]["va
```

(Again, exact path may change; we'll correct after we see one incoming payload.)

4. Execute and Test

1. In n8n workflow editor, click **"Execute workflow"** (or **"Test" / "Listen for webhook"** if shown).
2. From your phone, send a WhatsApp message (any text) to the **test number** `+1 555 169 2793`.
3. Watch n8n:
   - A new execution should appear.
   - If the send node succeeds, you'll see a reply on your phone.

If:

- Trigger fires but send node fails → copy the error text and we'll adjust the "To" expression or Phone Number ID.
- Trigger does not fire → we may need to configure the Meta **Webhook** (Step 4 in the API Testing page) and point it to the n8n webhook URL.

Once you try this, paste any error message (or confirm it replied) so the paths can be corrected and then you'll be ready to move from echo to Excel upload + timetable generation over WhatsApp.

⁂

how do i name it

To name the workflow in n8n:

1. At the top center you see **"My workflow"** (next to "Personal").
2. Click directly on **"My workflow"**.
3. It will become editable; type:
   - `FFCS WhatsApp Echo`
4. Press **Enter**.

Now the workflow is named. Next:

- Click the **“Add first step…”** box in the middle to add the **WhatsApp Trigger** node as described earlier.

⁂

image.jpg

Exactly what is needed. Now:

1. In the right panel, click **“WhatsApp Business Cloud”** (the first option).
   This will place the **trigger node** on the canvas.
2. In the node’s settings:
   - **Credentials**: choose your `WhatsApp account`.
   - If there is a **Phone Number ID** field, put `839904665883864`.
   - Leave other fields default for now.
3. Click **Save** (top-right of workflow) so this node configuration is stored.

Next, add the send node:

- Click the **+** button to the right of the trigger node → add another **WhatsApp API** node (not trigger).
- Then you’ll configure it to send a reply (echo). When you’ve added that node, if you share a screenshot or say “send node added”, the exact expression paths for “To” and message body can be given based on the trigger’s output.

⁂

image.jpg

Nice—this shows the **WhatsApp Business Cloud** node palette.

1. Add the trigger

1. Under “Triggers”, click **“On messages”**.
2. This will place an **On messages** WhatsApp node on the canvas as the first step.
3. Click that node and in its **Credentials** dropdown choose your `WhatsApp account`.
4. If there is a **Phone Number ID** field, set it to `839904665883864`.
5. Save the workflow (top right).

2. Add the send (action) node

1. Hover over the trigger node; you should see a small **+** to the right. Click it.
2. Choose **WhatsApp Business Cloud** → **Actions** → **“Send message”**.
3. In this new action node:

- Credentials: `WhatsApp account`.
- Phone Number ID: `839904665883864` (if present).
- **To**: we'll use a simple test first: type your own WhatsApp number with country code (e.g., `+91xxxxxxxxxx`).
- Message type: **Text**.
- Text body: `Hi from n8n FFCS Echo`.

4. Click **Execute workflow**, then press the small **Execute node** button on the send node to test it.

If you receive `Hi from n8n FFCS Echo` on your phone, credentials and send are confirmed. Then the next step will be changing **To** to an expression that uses the sender's number from the trigger, turning it into a real echo bot.

⁂

image.jpg

Good, this is the **WhatsApp Trigger** node configuration.

1. Attach your credential

- Click the **"Credential to connect with"** dropdown.
- Choose your `WhatsApp account` credential.

Leave **Trigger On** as **Messages**.

Click **Save** on the node (or save the workflow).

2. Next: add send node

Now:

1. Close the node panel (click canvas background).
2. Hover over the **WhatsApp Trigger** node box on the canvas (it should now appear).
3. Click the small **+** that appears to the right of it.
4. In the node list, choose **WhatsApp Business Cloud** → **Actions** → **"Send message"**.

After you add that send node, configure it with:

- Credentials: `WhatsApp account`.
- Phone Number ID (if field exists): `839904665883864`.
- To: temporarily set to your own WhatsApp number `+91…`.
- Message text: `Hi from n8n FFCS Echo`.

Then execute that node to confirm sending works.

⁂

image.jpg

This is the **WhatsApp OAuth account** screen; that's not the one you want for Cloud API with an access token.

Close this dialog (X in top-right) and:

1. In the **WhatsApp Trigger** node's "Credential to connect with" dropdown, choose the credential named something like **"WhatsApp account"** that you already created using the access token and Business Account ID (WhatsApp API credential).
2. Do **not** create or select "WhatsApp OAuth API" here.

If the dropdown only shows this OAuth credential and not your token-based one, it means:

- You created the token credential under **WhatsApp API**, not WhatsApp Business Cloud.

Then:

1. Cancel the OAuth credential.
2. Go to **Credentials** (left sidebar) → verify that you have a credential of type **WhatsApp API** with your token and Business Account ID.
3. Back in the Trigger node, the credential dropdown should list that; select it.

Once selected successfully, the red warning icon will disappear. Then add the **Send message** action node as described earlier.

⁂

if i press + symbol i search for whatsapp and it shows whatsapp business cloud and if i click that under trigger coloumn there is an option for on messages if i click that means it opens whatsapp oauth api and ask me to create. help me

What you're seeing is actually correct:

- **Trigger node (On messages)** must use **WhatsApp OAuth account (client ID + client secret)**.
- **Action node (Send message)** uses **WhatsApp Business Cloud API (access token + business account ID)**.[620] [621] [622]

So you need **two** credentials, not one.

1. Why it opens “WhatsApp OAuth API”

When you add **WhatsApp Business Cloud** → **On messages**, n8n creates a **WhatsApp Trigger** node.
This node **always** asks for a **WhatsApp OAuth account** credential, so it automatically opens the “WhatsApp OAuth API” popup when no OAuth credential exists yet.[623] [620]

That’s expected behavior, not a mistake.

2. What you must fill there

In that popup (the one in your last screenshot):

- **Client ID** → from Meta app (App ID).[621] [620]
- **Client Secret** → from Meta app (App secret).[620] [621]

Steps (short):

- Go to **Meta for Developers** → **Your App** → **Basic settings**.
- Copy **App ID** → paste as Client ID.
- Copy **App secret** (click “Show”) → paste as Client Secret.
- Click **Save** in n8n.

After saving, select this OAuth credential in **“Credential to connect with”** of the **WhatsApp Trigger** node.

3. Then set up sending

Once the trigger credential is done:

1. Add a new node with **+** → **WhatsApp Business Cloud** → **Send message**.
2. That node will ask for **WhatsApp Business Cloud API** credential (access token style), where you already have:
    - **Access Token**
    - **Business Account ID**.[624] [620]

Now you have:

Node type	Credential type	Fields you fill	
On messages	**WhatsApp OAuth account**	Client ID, Client Secret	
Send message	**WhatsApp Business Cloud**	Access Token, Business Account ID	[620] [622]

If you tell what you already have (App ID, App secret, access token, BA ID), the next step is wiring “To” and “Message” so it becomes your FFCS echo bot.

⁂

i added on message. next?

Nice, trigger is on the canvas. Now wire the send step and make it echo.

1. Add the send message node

1. Hover over the **WhatsApp Trigger** node.
2. Click the small **+** that appears on its right side.
3. In the menu, search **"whatsapp"**.
4. Under **WhatsApp Business Cloud** → **Actions**, choose **"Send message"**.

This will create a second node connected to the trigger.

2. Configure the send node (first test)

Open the **Send message** node and set:

- **Credentials**: select your **WhatsApp Business Cloud API** credential (the one with Access Token + Business Account ID).[664] [665]
- **Phone Number ID**: `839904665883864`.
- **To**: your own WhatsApp number with country code, like `+91xxxxxxxxxx`.
- **Message type**: Text.
- **Text body**: `Hi from n8n FFCS Echo`.

Save the workflow, click **Execute workflow**, then execute just the **Send message** node once to confirm you receive that test message.

3. Turn it into an echo bot

After the test works:

1. Change **To** from a fixed number to an **expression**:
   - Click the gears icon next to **To** → **Add expression**.
   - Use:
     `{{ $json.entry[^48_0].changes[^48_0].value.messages[^48_0].from }}`
2. Change **Text body** to:
   - **Expression** → `{{ $json.entry[^48_0].changes[^48_0].value.messages[^48_0].text.body }}`.[666] [667]

Now activate the workflow. Any WhatsApp message you send to that number should be echoed back.

⁂

for send message, wh i should fill here

For this **Send message** node, fill it like this for a quick test:

1. Fields to fill now

- **Credential to connect with**: keep your **WhatsApp account** credential (Business Cloud API). [693]
- **Sender Phone Number (or ID)**: choose the phone number entry that corresponds to `839904665883864` (your WhatsApp number ID). [694] [693]
- **Recipient's Phone Number**: put your own WhatsApp number in full international format, for example:
  `+91xxxxxxxxxx`
- **MessageType**: `Text`.
- **Text Body**: `Hi from n8n FFCS Echo`.

Leave **Additional Fields** empty for now.

Then:

1. Click **Save** (top right of the workflow).
2. Click **Execute step** on this node.

You should receive `Hi from n8n FFCS Echo` on your WhatsApp. Once this works, the next step is to change **Recipient's Phone Number** and **Text Body** to expressions that read from the trigger so it becomes a real echo bot.

⁂

image.jpg

The error is coming from the **WhatsApp Trigger**, not the Send node: "WhatsApp Trigger: Invalid parameter".

1. Fix the trigger first

Open the **WhatsApp Trigger** node and check:

- **Credential to connect with**: must be your **WhatsApp OAuth account** (Client ID + Client Secret from your Meta app). [721]
- **Trigger on**: `Messages` only.
- In the node's **Settings** → **Webhook URLs**, the **Production URL** must be the same as the one configured in your WhatsApp app webhook in Meta, and verification completed. [722] [723]

If any of these is wrong, Meta returns "invalid parameter", and n8n shows that message.

After fixing, click:

1. **Save** the workflow.
2. In the Trigger node, click **Test this trigger** and then send a WhatsApp message to your test number to see if data appears in the OUTPUT panel.[724]

2. Then test the Send message node

Once the trigger works:

- Keep **Sender Phone Number (or ID)** as your test number (`1555… Test Number`).
- **Recipient's Phone Number**: your full WhatsApp number `+91 78741 41556` (no spaces).
- **Text Body**: `Hi from n8n FFCS Echo`.

Now click **Execute step** on the **Send message** node.
If you still get an error, open **Show Details** on the red toast and paste the exact error text here so the wrong parameter can be identified precisely.

⁂

image.jpg

Trigger node looks good now; next step is to **test it and then wire the echo**.

1. Test the trigger

1. Click **Test this trigger** on the right side of the Trigger node panel.
2. When it shows "Listening for events…", send a WhatsApp message (any text) to your test number (the one from Meta).
3. If everything is correct, the **OUTPUT** panel on the right should fill with a JSON payload containing `entry`, `changes`, `messages`, etc.[752] [753]

If you see data there, your webhook + OAuth are working.

2. Make Send node use trigger data (echo)

Open the **Send message** node and set:

- **Recipient's Phone Number** → click gear icon → **Add Expression** → paste:
  `{{ $json.entry[^51_0].changes[^51_0].value.messages[^51_0].from }}`
- **Text Body** → click gear icon → **Add Expression** → paste:
  `{{ $json.entry[^51_0].changes[^51_0].value.messages[^51_0].text.body }}`[754] [752]

Leave **Sender Phone Number (or ID)** as the test number, same as before.

3. Run the full workflow

1. Click **Activate** (top-right) so the workflow listens automatically.
2. Send a WhatsApp message to the test number again.

You should now receive back exactly the same text you sent (your FFCS echo).

⁂

give me an entire n8n layout

Here is a simple **two-node n8n layout** for your FFCS WhatsApp Echo bot.

1. Nodes on the canvas

- **Node 1: WhatsApp Trigger** (name it `WhatsApp Trigger`).
- **Node 2: WhatsApp Business Cloud → Send message** (name it `Send message`).[783] [784]

Connect: **WhatsApp Trigger → Send message** with a straight line.

2. WhatsApp Trigger configuration

In the **WhatsApp Trigger** node:

- **Credential to connect with**: `WhatsApp OAuth account` (Client ID + Client Secret from your Meta app).
- **Trigger On**: `Messages` only.
- Leave **Options** empty.[785] [783]

After saving, click **Test this trigger**, send a WhatsApp message to your test number, and confirm you see JSON output on the right.

3. Send message node configuration

In the **Send message** node:

- **Credential to connect with**: your **WhatsApp account** (Business Cloud API, with access token + business account ID).[784] [785]
- **Resource**: `Message`.
- **Operation**: `Send`.
- **Sender Phone Number (or ID)**: choose your WhatsApp test number entry (`1555… Test Number` in your screenshot).

Now set these two key fields as **expressions** so it becomes an echo:

- **Recipient's Phone Number**
  - Click gear → **Add expression** → paste:
    `{{ $json.entry[^52_0].changes[^52_0].value.messages[^52_0].from }}`

- **MessageType**: `Text`.
- **Text Body**
  - Click gear → **Add expression** → paste:
    `{{ $json.entry[^52_0].changes[^52_0].value.messages[^52_0].text.body }}`[786] [783]

No additional fields needed.

4. Workflow settings

- Name: `FFCS WhatsApp Echo`.
- Save the workflow.
- Click **Activate** so it listens continuously.[786]

Now when you send any message to your WhatsApp test number:

- Node 1 receives it.
- Node 2 sends the same text back to the sender's number, giving you a working echo bot.

⁂

image.jpg

Your layout looks **perfect**: WhatsApp Trigger → AI Agent → Send message, with model, memory, and tool connected.

To finish it so WhatsApp users talk to the AI:

1. WhatsApp Trigger → AI Agent

In **AI Agent** node:

- **Chat Input**: expression that reads WhatsApp text, e.g.
  `{{ $json.entry[^53_0].changes[^53_0].value.messages[^53_0].text.body }}`[831] [832]
- **Sender ID / User ID** (for memory):
  `{{ $json.entry[^53_0].changes[^53_0].value.messages[^53_0].from }}`

Set:

- **Chat Model**: connect your **OpenAI Chat Model** node.
- **Memory**: connect **Simple Memory**.
- **Tool**: connect **Code Tool** only if you need custom logic.[833]

2. AI Agent → Send message

In **Send message** node:

- **Credential**: your WhatsApp Business Cloud API credential.
- **Sender Phone Number (or ID)**: your WhatsApp number ID (same as before).[834] [835]

- **Recipient's Phone Number** → expression:
  `{{ $json.entry[^53_0].changes[^53_0].value.messages[^53_0].from }}` (so reply to the same user; make sure node input is **AI Agent** output carrying original trigger JSON).[831]
- **MessageType**: `Text`.
- **Text Body** → expression using AI Agent output, e.g.:
  `{{ $json.data[^53_0].answer || $json.answer }}`
  (check the AI Agent output once by executing it and adjust the path to the `answer` field).
  [836] [837]

3. Test flow

1. **Test this trigger** on WhatsApp Trigger.
2. Send a WhatsApp message to your number.
3. Verify AI Agent output shows an `answer` field.
4. Activate workflow; send another message and confirm you receive the AI reply on WhatsApp.

If you share the JSON from the AI Agent output panel once, the exact expression for **Text Body** can be given precisely.

⁂

check wh and all i did and need to change.

You have the structure right; only a few mappings need to change so WhatsApp → AI Agent → Code Tool → Send message line up correctly.

WhatsApp Trigger

- Keep **Credential** = WhatsApp OAuth account and **Trigger On** = Messages (this is fine).
- In Meta, be sure the webhook sends `messages` and `message_postbacks` so you always get `entry[^54_0].changes[^54_0].value.messages[^54_0]`.[867] [868]

AI Agent

Change how it reads the user message and identifies the session:

- **Source for Prompt (User Message)**: set to **Connected Chat Trigger Node** (you already did, good).
- **Prompt (User Message)**: instead of `{{ $json.chatInput }}`, use the WhatsApp text:
  `{{ $json.entry[^54_0].changes[^54_0].value.messages[^54_0].text.body }}`[867]
- To make memory follow one WhatsApp user, add a new **Option** → **User ID** and set expression:
  `{{ $json.entry[^54_0].changes[^54_0].value.messages[^54_0].from }}`.[869]

Keep **Chat Model** connected to your OpenAI Chat Model, **Memory** to Simple Memory, and **Tool** to Code Tool (all good).

Simple Memory

Right now it reads `{{ $json.sessionId }}`, which you never set.

- **Session ID**: choose **Connected Chat Trigger Node** (ok).
- **Session Key From Previous Node**: change to the same user ID expression so every phone number is its own session:
  `{{ $json.entry[^54_0].changes[^54_0].value.messages[^54_0].from }}`.[869]

Context window length `5` is fine.

OpenAI Chat Model

This is already correct:

- **Model**: `gpt-4.1-mini` with Responses API.
- No changes needed here.[870]

Code Tool

You are using it as a **tool** for the agent (good). For that to work:

- The function must return a **string** (or serialised JSON string) because the agent expects a textual tool result.[870]
- At the end of your JavaScript code, ensure something like:

```
return JSON.stringify({
  status: 'ok',
  timetables: results
});
```

- In the **Description** field, clearly say what the tool does and what input format it expects (you already wrote a good description).

You do **not** need to call the Code Tool manually; the AI Agent will decide when to call it.

Send message

Wire it to use the AI answer and reply to the sender:

- **Credential**: WhatsApp Business Cloud API (unchanged).
- **Sender Phone Number (or ID)**: your WhatsApp number ID (same as before).
- **Recipient's Phone Number** → expression (from WhatsApp Trigger data that the agent passes through):
  `{{ $json.entry[^54_0].changes[^54_0].value.messages[^54_0].from }}`[867]
- **MessageType**: Text.

- **Text Body** → expression using AI Agent output:
  After you run AI Agent once, inspect its OUTPUT; usually the answer is in `data[^54_0].answer`. Try:
  `{{ $json.data[^54_0].answer || $json.answer }}`.[871] [872]

If that expression shows `undefined` in the preview, send one sample message, open the **AI Agent** → **OUTPUT** panel, and tell the exact JSON path you see for the final answer so it can be adjusted precisely.

After these changes, activate the workflow and send a WhatsApp message; it should respond with the AI's timetable guidance instead of just echoing.

⁂

check this too

Your Send message node is very close; two fields need more precise expressions so it reliably replies to the right person with the AI answer.

Recipient's Phone Number

Right now you use `{{ $json.from }}`, but that field does not exist by default.
Use the WhatsApp Trigger's sender number path instead:

- Click the gear next to **Recipient's Phone Number** → **Add expression** and set:

```
{{ $json.entry[^55_0].changes[^55_0].value.messages[^55_0].from }}
```

This is where the WhatsApp Trigger stores the user's phone number.[907] [908]

Text Body (AI answer)

Currently `{{ $json.output }}` points to a non-standard field.
Inspect one sample **AI Agent** → **OUTPUT**; in most n8n AI Agent setups the main answer is in `data[^55_0].answer`.[909] [910]

So set **Text Body** expression to:

```
{{ $json.data[^55_0].answer || $json.answer }}
```

If you test once and the output JSON shows a different path (for example `result.answer`), adjust the expression to that exact path.

After these two changes:

1. Execute **WhatsApp Trigger** in "Test this trigger" mode and send a WhatsApp message.
2. Execute **AI Agent** and confirm `data[^55_0].answer` contains text.

3. Execute **Send message** (or activate the workflow) and you should receive that AI answer back on WhatsApp.

⁂

i m using comet to automate my work and it asked this: You are on the right track; only a few expressions need to be updated so the WhatsApp Trigger → AI Agent → Code Tool → Send Message pipeline lines up and keeps per-user memory.

WhatsApp Trigger
For the trigger node, keep what you already have and double-check the JSON structure used everywhere else.
Credential: WhatsApp OAuth / Business Cloud credential (unchanged).
Trigger On: Messages (correct).
In Meta webhook, ensure messages and message_postbacks are subscribed so the payload always has entry[0].changes[0].value.messages[0].
The key fields you will use in expressions later are:
Text: {{ $json.entry[0].changes[0].value.messages[0].text.body }}.
Sender ID (phone): {{ $json.entry[0].changes[0].value.messages[0].from }}.
AI Agent node
Update the AI Agent so it reads the correct WhatsApp text and identifies the user for memory.
Source for Prompt (User Message): Connected Chat Trigger Node (keep as is).
Prompt (User Message):
{{ $json.entry[0].changes[0].value.messages[0].text.body }}.
Add Option → User ID and set:
{{ $json.entry[0].changes[0].value.messages[0].from }}.
This makes each phone number its own chat session so the Simple Memory node can keep context per user.
Keep:
Chat Model: your OpenAI Chat Model (gpt-4.1-mini, Responses API).
Memory: Simple Memory.
Tool: Code Tool.
Simple Memory node
Right now you have {{ $json.sessionId }}, which is never set, so memory cannot follow a user.
Set:
Session ID → Connected Chat Trigger Node.
Session Key From Previous Node → same “from” path used in AI Agent:
{{ $json.entry[0].changes[0].value.messages[0].from }}.
This ensures a consistent key between AI Agent and Simple Memory for each WhatsApp number.
Code Tool node
Because the Code Tool is a tool for the AI Agent, it must return a plain string.
At the end of your JavaScript, do something like:

```
js
return JSON.stringify({
status: 'ok',
timetables: results,
});
```

The Description should clearly tell the agent what the tool does and what input it expects; your existing description is already in the right direction.
You do not need to manually execute this node during normal runs; the agent calls it when useful.
Send Message node
This is where most alignment issues usually appear: you must pull the recipient from the trigger and the text body from the AI Agent output.

1. Recipient’s Phone Number
   Do not use {{ $json.from }} because that field is not present by default in the trigger output.
   Instead, set the expression to the same sender path:
   ```
   js
   {{ $json.entry[0].changes[0].value.messages[0].from }}
   ```

This reliably targets the original WhatsApp sender.
2) Text Body (AI answer)
The AI Agent usually outputs its main answer under data[0].answer.
In Text Body, set the expression to:
```
js
{{ $json.data[0].answer || $json.answer }}
```

If preview shows undefined, run the AI Agent once, open its OUTPUT, and look for the actual field path (for example, result.answer or similar), then adapt the expression to that path.
Keep other fields:
Credential: WhatsApp Business Cloud API.
Sender Phone Number (or ID): your WhatsApp number ID from Meta.
MessageType: Text.
Test sequence to confirm
To verify everything end-to-end:
Use “Test this trigger” on WhatsApp Trigger and send a message from your phone; confirm you see entry[0].changes[0].value.messages[0].text.body and .from.
Run the AI Agent once and inspect its output to confirm where the answer lives (ideally data[0].answer).
Run the Send Message node (or activate the workflow) and send a new WhatsApp message; you should receive the AI’s timetable response instead of an echo.
If you paste one sample AI Agent OUTPUT JSON, the exact Text Body expression can be adjusted to your instance’s structure with full precision.

Everything that text suggests is correct for your setup; you just need to apply those exact expressions in each node.

WhatsApp Trigger

No change needed now, just remember these two paths:

- **User message text**:
  `{{ $json.entry[^56_0].changes[^56_0].value.messages[^56_0].text.body }}`
- **Sender phone (user id)**:
  `{{ $json.entry[^56_0].changes[^56_0].value.messages[^56_0].from }}`[946] [947]

You will reuse them in AI Agent, Simple Memory, and Send message.

AI Agent

Update:

- **Prompt (User Message)** → expression:
  `{{ $json.entry[^56_0].changes[^56_0].value.messages[^56_0].text.body }}`
- Add **Option** → **User ID** → expression:
  `{{ $json.entry[^56_0].changes[^56_0].value.messages[^56_0].from }}`[948] [949]

Keep Chat Model, Memory, and Tool connections as you already have.

Simple Memory

Change:

- **Session ID** → `Connected Chat Trigger Node`.
- **Session Key From Previous Node** → expression:
  `{{ $json.entry[^56_0].changes[^56_0].value.messages[^56_0].from }}`[949]

Now the same WhatsApp number always maps to the same memory key.

Code Tool

At the bottom of your JS, make sure you return a **string**, e.g.:

```
return JSON.stringify({
  status: 'ok',
  timetables: results,
});
```

Leave the description explaining what input it expects.

Send message

Fix both expressions:

- **Recipient's Phone Number** → expression:
  `{{ $json.entry[^56_0].changes[^56_0].value.messages[^56_0].from }}`[946]

- **Text Body** → expression (temporary best guess):
  `{{ $json.data[^56_0].answer || $json.answer }}`[950] [948]

If, after one test, the message body is empty or “undefined”, open **AI Agent** → **OUTPUT**, copy the JSON, and share it; then the Text Body expression can be adjusted precisely to the actual field name in your instance.

⁂

1. https://n8n.io/integrations/whatsapp-business-cloud/
2. https://www.heltar.com/blogs/how-to-integrate-whatsapp-api-with-n8n
3. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
4. https://www.youtube.com/watch?v=D9JF-aMSCNs
5. https://n8n.io/workflows/3586-ai-powered-whatsapp-chatbot-for-text-voice-images-and-pdfs-with-memory/
6. https://www.youtube.com/watch?v=DU2hwUsMBBM
7. https://community.n8n.io/t/ow-to-read-a-text-file-or-spreadsheet-sent-via-whatsapp-in-n8n/85790
8. https://n8n.io/workflows/4213-automatic-media-download-from-whatsapp-business-messages-with-http-storage/
9. https://n8n.io/workflows/2162-whatsapp-starter-workflow/
10. https://n8n.io/integrations/pdfmonkey/and/whatsapp-business-cloud/
11. https://n8n.io/workflows/7288-generate-and-share-professional-pdfs-with-openai-google-docs-and-slack/
12. https://www.reddit.com/r/CodingHelp/comments/w3eqwg/how_to_send_the_pdf_files_directly_to_whatsapp/
13. https://wbiztool.com/docs/n8n-integration/
14. https://aiagencyplus.com/n8n-whatsapp-integration-complete-guide/
15. https://n8n.io/workflows/4827-ai-powered-whatsapp-chatbot-for-text-voice-images-and-pdf-with-rag/
16. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
17. https://www.youtube.com/watch?v=XJd41xmXoJM
18. https://www.reddit.com/r/n8n/comments/1k0p3l0/whatsapp_via_twilio_and_n8n_integration_how_to/
19. https://www.reddit.com/r/n8n/comments/1k0490p/free_whatsapp_api_for_n8n/
20. https://www.reddit.com/r/n8n/comments/1iuuqoc/which_tool_to_use_for_integrations_with_whatsapp/
21. https://www.youtube.com/watch?v=Lo4wrGoWEjk
22. https://vitspot.com/posts/vit-ffcs/
23. https://devendraap.github.io/Time-table-creator/
24. https://ffcsonthego.vatz88.in
25. http://ffcsvit.blogspot.com/2012/03/in-previous-post-we-discussed-about.html
26. https://vit.ac.in/sites/default/files/academic/Academic-Regulations.pdf

27. https://www.studocu.com/in/document/vellore-institute-of-technology/environmental-biotechnology/academic-regulations-for-vit-university/21632569
28. https://www.youtube.com/watch?v=xN_oSF3ED08
29. https://www.ijfmr.com/papers/2025/6/63501.pdf
30. https://www.ijfmr.com/research-paper.php?id=63501
31. https://n8n.io/integrations/whatsapp-business-cloud/
32. https://n8n.io/workflows/3586-ai-powered-whatsapp-chatbot-for-text-voice-images-and-pdfs-with-memory/
33. https://community.n8n.io/t/ow-to-read-a-text-file-or-spreadsheet-sent-via-whatsapp-in-n8n/85790
34. https://n8n.io/workflows/4213-automatic-media-download-from-whatsapp-business-messages-with-http-storage/
35. https://blog.n8n.io/multi-agent-systems/
36. https://blog.n8n.io/ai-agents-examples/
37. https://blog.n8n.io/ai-agentic-workflows/
38. https://n8n.io/workflows/2942-ai-agent-for-top-n8n-creators-leaderboard-reporting/
39. https://n8n.io/workflows/7288-generate-and-share-professional-pdfs-with-openai-google-docs-and-slack/
40. https://n8n.io/integrations/pdfmonkey/and/whatsapp-business-cloud/
41. https://www.youtube.com/watch?v=DU2hwUsMBBM
42. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
43. https://www.perplexity.ai/search/32ab1252-93a6-4337-843e-d572975741d7
44. https://www.perplexity.ai/search/f2bed815-2d94-4813-af1f-71bf971a78d5
45. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
46. https://dev.to/gaurang_parante/how-to-upload-an-excel-file-in-nodejs-using-express-and-multer-ici
47. https://www.geeksforgeeks.org/node-js/how-to-read-and-write-excel-file-in-node-js/
48. https://moldstud.com/articles/p-best-practices-for-structuring-your-first-mern-project-a-comprehensive-guide
49. https://dev.to/kingsley/mern-stack-project-structure-best-practices-2adk
50. https://stackoverflow.com/questions/78771089/how-to-upload-excel-file-in-node-js-using-express-and-multer
51. https://www.linkedin.com/pulse/how-upload-parse-excel-files-nodejs-express-vivek-neupane-iu9hc
52. https://stackoverflow.com/questions/70562438/how-to-upload-and-read-excel-file-in-nodejs
53. https://www.zignuts.com/blog/mern-stack-2025-full-stack-apps-guide
54. https://www.guvi.in/blog/mern-stack-best-practices/
55. https://www.youtube.com/watch?v=F9gB5b4jgOI
56. https://www.tuvoc.com/blog/complete-guide-mern-stack-development/
57. https://www.npmjs.com/package/read-excel-file
58. https://javascript.plainenglish.io/the-mern-stack-roadmap-that-actually-works-in-2025-c239feaa4532

59. https://stackoverflow.com/questions/28860728/reading-excel-file-using-node-js
60. https://gigaversity.in/blog/mern-stack-development-guide-2025
61. https://www.bezkoder.com/node-js-express-file-upload/
62. https://www.youtube.com/watch?v=c9fvwC3OBjI
63. https://www.reddit.com/r/node/comments/y1x56v/im_building_a_backend_server_that_parses_an_excel/
64. https://www.geeksforgeeks.org/mern/mern-stack-projects/
65. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
66. https://strapi.io/blog/mern-stack-guide-components-setup-best-practices
67. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
68. https://dev.to/gaurang_parante/how-to-upload-an-excel-file-in-nodejs-using-express-and-multer-ici
69. https://www.npmjs.com/package/read-excel-file
70. https://www.linkedin.com/pulse/how-upload-parse-excel-files-nodejs-express-vivek-neupane-iu9hc
71. https://stackoverflow.com/questions/70562438/how-to-upload-and-read-excel-file-in-nodejs
72. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
73. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
74. https://gigaversity.in/blog/mern-stack-development-guide-2025
75. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
76. image.jpg
77. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
78. image.jpg
79. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
80. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
81. image.jpg
82. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
83. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
84. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
85. https://dev.to/gaurang_parante/how-to-upload-an-excel-file-in-nodejs-using-express-and-multer-ici
86. https://www.npmjs.com/package/read-excel-file
87. image.jpg

88. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
89. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
90. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
91. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
92. https://strapi.io/blog/mern-stack-guide-components-setup-best-practices
93. https://dev.to/kingsley/mern-stack-project-structure-best-practices-2adk
94. https://dev.to/gaurang_parante/how-to-upload-an-excel-file-in-nodejs-using-express-and-multer-ici
95. https://www.bezkoder.com/node-js-express-file-upload/
96. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
97. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
98. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
99. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
100. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
101. image.jpg
102. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
103. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
104. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
105. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
106. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
107. image.jpg
108. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
109. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
110. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
111. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg

112. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
113. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
114. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
115. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
116. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
117. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
118. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
119. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
120. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
121. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
122. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
123. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
124. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
125. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
126. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
127. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
128. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
129. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
130. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
131. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
132. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
133. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg

134. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
135. image.jpg
136. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
137. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
138. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
139. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
140. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
141. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
142. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
143. https://ijetrm.com/issues/files/Jun-2025-05-1749134157-JUNE11.pdf
144. https://www.ijfmr.com/papers/2025/6/63501.pdf
145. https://vitspot.com/posts/vit-ffcs/
146. https://devendraap.github.io/Time-table-creator/
147. http://ffcsvit.blogspot.com/2012/03/in-previous-post-we-discussed-about.html
148. https://ijarbest.com/conference/spcl15/662
149. https://ijarcce.com/wp-content/uploads/2023/05/IJARCCE.2023.124157.pdf
150. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
151. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
152. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
153. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
154. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
155. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
156. https://www.ssgmce.ac.in/uploads/UG_Projects/cse/202425/Proj.Gr.-09-Project_Report_2024-25.pdf
157. https://stackoverflow.com/questions/35838005/backtracking-solution-hint-for-generating-schedule
158. https://stackoverflow.com/questions/76075920/i-want-to-create-a-time-slot-without-overlapping-any-existing-time-slots-in-mong
159. https://www.atlantis-press.com/article/125993049.pdf
160. https://www.unitime.org/papers/phd05.pdf

161. https://github.com/ori88c/non-overlapping-periodic-job-scheduler
162. https://www.scirp.org/journal/paperinformation?paperid=73004
163. https://ijsrem.com/download/constraint-driven-timetabling-with-real-time-updates-design-and-implementation/
164. https://www.youtube.com/watch?v=3S0z_Wn81wY
165. https://www.irjet.net/archives/V10/i11/IRJET-V10I1109.pdf
166. https://rjpn.org/ijcspub/papers/IJCSP25A1130.pdf
167. https://demo.mobiscroll.com/scheduler/prevent-double-booking-events
168. https://www.ijresm.com/Vol.3_2020/Vol3_Iss3_March20/IJRESM_V3_I3_100.pdf
169. https://www.techrxiv.org/users/908376/articles/1282333/master/file/data/Automatic_Timetable_Generation_Research_Paper (7)/Automatic_Timetable_Generation_Research_Paper (7).pdf
170. https://code.daypilot.org/13165/javascript-scheduler-disabling-custom-date-time-ranges
171. https://github.com/Parthiee/Timetable-Generator
172. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
173. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
174. https://www.ijfmr.com/papers/2025/6/63501.pdf
175. https://ijarbest.com/conference/spcl15/662
176. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
177. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
178. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
179. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
180. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
181. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
182. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
183. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
184. https://www.ijfmr.com/papers/2025/6/63501.pdf
185. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
186. image.jpg
187. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg

188. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
189. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
190. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
191. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
192. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
193. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
194. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
195. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
196. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
197. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
198. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
199. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
200. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
201. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
202. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
203. https://n8n.io/integrations/whatsapp-business-cloud/
204. https://n8n.io/workflows/3586-ai-powered-whatsapp-chatbot-for-text-voice-images-and-pdfs-with-memory/
205. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
206. https://www.heltar.com/blogs/how-to-integrate-whatsapp-api-with-n8n
207. https://community.n8n.io/t/ow-to-read-a-text-file-or-spreadsheet-sent-via-whatsapp-in-n8n/85790
208. https://n8n.io/workflows/4213-automatic-media-download-from-whatsapp-business-messages-with-http-storage/
209. https://n8n.io/workflows/7288-generate-and-share-professional-pdfs-with-openai-google-docs-and-slack/
210. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
211. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg

212. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
213. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
214. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
215. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
216. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
217. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
218. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
219. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
220. https://www.heltar.com/blogs/how-to-integrate-whatsapp-api-with-n8n
221. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
222. https://www.youtube.com/watch?v=LekI91lSf_8&vl=en
223. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
224. https://community.n8n.io/t/how-to-get-whatsapp-business-cloud-webhook-to-trigger/28977
225. https://n8n.io/integrations/whatsapp-business-cloud/
226. https://hackceleration.com/whatsapp-n8n/
227. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
228. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
229. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
230. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
231. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
232. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
233. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
234. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
235. https://n8n.io/workflows/3586-ai-powered-whatsapp-chatbot-for-text-voice-images-and-pdfs-with-memory/
236. https://www.bitcot.com/building-custom-whatsapp-ai-agents-using-n8n-and-openai/
237. https://ayn8n.com/workflows/automated-whatsapp-business-media-download-and-http-storage-workflow

238. https://www.voiceflow.com/blog/n8n-whatsapp-chatbot
239. https://n8n.io/workflows/4213-automatic-media-download-from-whatsapp-business-messages-with-http-storage/
240. https://wassenger.com/blog/create-a-full-featured-whatsapp-ai-chatbot-with-n8n
241. https://www.youtube.com/watch?v=DU2hwUsMBBM
242. https://community.n8n.io/t/whatsapp-download-media-node/92840
243. https://n8n.io/workflows/2465-building-your-first-whatsapp-chatbot/
244. https://flows8.com/items/automatic-media-download-from-whatsapp-business-messages-with-http-storage/1905
245. https://n8n.io/workflows/4827-ai-powered-whatsapp-chatbot-for-text-voice-images-and-pdf-with-rag/
246. https://www.hostinger.com/in/tutorials/how-to-integrate-n8n-with-whatsapp
247. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
248. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
249. https://community.n8n.io/t/how-to-get-whatsapp-business-cloud-webhook-to-trigger/28977
250. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
251. https://www.youtube.com/watch?v=LekI91lSf_8&vl=en
252. https://www.hostinger.com/in/tutorials/how-to-integrate-n8n-with-whatsapp
253. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
254. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
255. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
256. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
257. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
258. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
259. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
260. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
261. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
262. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
263. https://community.n8n.io/t/how-to-get-whatsapp-business-cloud-webhook-to-trigger/28977
264. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
265. https://www.hostinger.com/in/tutorials/how-to-integrate-n8n-with-whatsapp
266. https://hackceleration.com/whatsapp-n8n/

267. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
268. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
269. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
270. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
271. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
272. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
273. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
274. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
275. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
276. https://community.n8n.io/t/how-to-get-whatsapp-business-cloud-webhook-to-trigger/28977
277. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
278. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
279. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
280. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
281. image.jpg
282. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
283. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
284. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
285. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
286. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
287. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
288. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
289. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
290. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg

291. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
292. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
293. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
294. image.jpg
295. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
296. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
297. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
298. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
299. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
300. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
301. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
302. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
303. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
304. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
305. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
306. image.jpg
307. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
308. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
309. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
310. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
311. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
312. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
313. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
314. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/

315. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
316. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
317. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
318. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
319. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
320. image.jpg
321. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
322. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
323. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
324. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
325. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
326. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
327. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
328. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
329. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
330. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
331. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
332. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
333. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
334. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
335. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
336. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
337. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg

338. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
339. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
340. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
341. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
342. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
343. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
344. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
345. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
346. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
347. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
348. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
349. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
350. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
351. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
352. image.jpg
353. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
354. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
355. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
356. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
357. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
358. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
359. image.jpg
360. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
361. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg

362. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
363. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
364. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
365. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
366. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
367. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
368. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
369. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
370. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
371. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
372. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
373. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
374. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
375. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
376. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
377. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
378. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
379. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
380. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
381. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
382. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
383. image.jpg
384. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg

385. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
386. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
387. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
388. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
389. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
390. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
391. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
392. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
393. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
394. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
395. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
396. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
397. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
398. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
399. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
400. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
401. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
402. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
403. image.jpg
404. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
405. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
406. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
407. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg

408. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
409. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
410. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
411. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
412. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
413. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
414. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
415. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
416. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
417. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
418. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
419. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
420. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
421. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
422. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
423. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
424. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
425. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
426. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
427. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
428. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
429. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
430. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
431. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/

432. https://hackceleration.com/whatsapp-n8n/
433. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
434. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
435. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
436. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
437. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
438. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
439. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
440. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
441. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
442. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
443. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
444. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
445. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
446. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
447. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
448. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
449. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
450. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
451. https://www.hostinger.com/in/tutorials/how-to-integrate-n8n-with-whatsapp
452. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
453. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
454. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg

455. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
456. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
457. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
458. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
459. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
460. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
461. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
462. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
463. image.jpg
464. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
465. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
466. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
467. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
468. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
469. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
470. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
471. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
472. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
473. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
474. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
475. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
476. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
477. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg

478. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
479. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
480. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
481. image.jpg
482. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
483. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
484. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
485. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
486. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
487. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
488. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
489. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
490. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
491. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
492. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
493. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
494. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
495. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
496. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
497. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
498. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
499. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
500. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg

501. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
502. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
503. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
504. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
505. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
506. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
507. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
508. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
509. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
510. image.jpg
511. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
512. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
513. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
514. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
515. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
516. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
517. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
518. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
519. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
520. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
521. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
522. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg

523. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
524. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
525. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
526. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
527. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
528. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
529. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
530. image.jpg
531. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
532. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
533. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
534. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
535. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
536. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
537. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
538. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
539. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
540. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
541. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
542. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
543. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
544. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg

545. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
546. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
547. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
548. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
549. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
550. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
551. image.jpg
552. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
553. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
554. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
555. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
556. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
557. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
558. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
559. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
560. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
561. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
562. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
563. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
564. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
565. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg
566. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg

567. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
568. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
569. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
570. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
571. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
572. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
573. image.jpg
574. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
575. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
576. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
577. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
578. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
579. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
580. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
581. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
582. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
583. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
584. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
585. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
586. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
587. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg
588. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg

589. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
590. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
591. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
592. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
593. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
594. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
595. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
596. image.jpg
597. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
598. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
599. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
600. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
601. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
602. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
603. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
604. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
605. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
606. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
607. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
608. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
609. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
610. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg

611. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg
612. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3f51479c-fc4c-41fb-8bbb-67c50aefcafa/image.jpg
613. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
614. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
615. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
616. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
617. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
618. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
619. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
620. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
621. https://www.hostinger.com/in/tutorials/how-to-integrate-n8n-with-whatsapp
622. https://www.youtube.com/watch?v=KwklHSYoc3E
623. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
624. https://community.n8n.io/t/when-i-try-to-execute-a-whatsapp-trigger-node-it-says-this-needs-to-be-triggered-with-access-token-but-the-oauth-credentials-are-taken-for-connection-test-how-do-i-fix-this-issue/213760
625. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
626. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
627. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
628. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
629. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
630. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
631. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
632. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
633. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg

634. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
635. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
636. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
637. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
638. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
639. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
640. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
641. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg
642. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg
643. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3f51479c-fc4c-41fb-8bbb-67c50aefcafa/image.jpg
644. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5a618c0b-ac35-454d-a922-0c5d8b7c4159/image.jpg
645. https://community.n8n.io/t/how-do-i-add-verification-token-for-whatsapp-business-cloud-trigger/93070
646. https://n8n.io/integrations/whatsapp-business-cloud/
647. https://community.n8n.io/t/access-token-expired-continuously/98749
648. https://www.youtube.com/watch?v=ZrhTQle55LQ
649. https://www.youtube.com/watch?v=A0OwvNOLNlw
650. https://hackceleration.com/whatsapp-n8n/
651. https://www.reddit.com/r/n8n/comments/1m22hyx/how_to_automate_credential_setup_in_n8n/
652. https://www.youtube.com/watch?v=g4BmhzK3fK8
653. https://www.linkedin.com/posts/shiva-ganti-ab452b312_n8n-whatsapp-integration-yes-please-activity-7319819631676821505-EaH9
654. https://nocodestartup.io/en/step-by-step-to-integrate-official-whatsapp-api-in-n8n-2/
655. https://community.n8n.io/t/how-to-setup-whatsapp-trigger-node-for-self-hosted-n8n-using-ngrok/203679
656. https://community.n8n.io/t/how-to-create-whatsapp-credentials-to-offer-to-a-company/147141
657. https://community.n8n.io/t/whatsapp-trigger-not-receiving-real-messages/239346
658. https://www.youtube.com/watch?v=5k7xhXZPu-k
659. https://github.com/n8n-io/n8n/issues/19158
660. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg

661. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
662. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
663. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
664. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
665. https://www.hostinger.com/in/tutorials/how-to-integrate-n8n-with-whatsapp
666. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
667. https://community.n8n.io/t/whatsapp-trigger-not-receiving-real-messages/239346
668. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
669. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
670. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
671. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
672. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
673. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
674. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
675. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
676. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
677. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
678. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
679. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
680. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
681. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
682. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
683. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
684. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg

685. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg
686. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3f51479c-fc4c-41fb-8bbb-67c50aefcafa/image.jpg
687. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5a618c0b-ac35-454d-a922-0c5d8b7c4159/image.jpg
688. image.jpg
689. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
690. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
691. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
692. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
693. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
694. https://www.hostinger.com/in/tutorials/how-to-integrate-n8n-with-whatsapp
695. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
696. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
697. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
698. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
699. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
700. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
701. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
702. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
703. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
704. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
705. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
706. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
707. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg

708. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
709. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg
710. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg
711. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3f51479c-fc4c-41fb-8bbb-67c50aefcafa/image.jpg
712. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5a618c0b-ac35-454d-a922-0c5d8b7c4159/image.jpg
713. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ab47c9f-0184-4624-b82b-930763a001a2/image.jpg
714. image.jpg
715. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
716. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
717. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
718. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
719. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
720. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
721. https://docs.n8n.io/integrations/builtin/credentials/whatsapp/
722. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
723. https://community.n8n.io/t/how-to-setup-whatsapp-trigger-node-for-self-hosted-n8n-using-ngrok/203679
724. https://community.n8n.io/t/whatsapp-trigger-not-receiving-real-messages/239346
725. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
726. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
727. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
728. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
729. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
730. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
731. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg

732. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
733. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
734. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
735. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
736. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
737. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
738. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
739. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
740. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
741. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg
742. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg
743. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3f51479c-fc4c-41fb-8bbb-67c50aefcafa/image.jpg
744. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5a618c0b-ac35-454d-a922-0c5d8b7c4159/image.jpg
745. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ab47c9f-0184-4624-b82b-930763a001a2/image.jpg
746. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/7cf0482b-e62b-4f22-875f-3663f12e80d2/image.jpg
747. image.jpg
748. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
749. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
750. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
751. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
752. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
753. https://community.n8n.io/t/whatsapp-trigger-not-receiving-real-messages/239346
754. https://hackceleration.com/whatsapp-n8n/
755. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg

756. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
757. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
758. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
759. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
760. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
761. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
762. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
763. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
764. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
765. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
766. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
767. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
768. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
769. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
770. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg
771. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg
772. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3f51479c-fc4c-41fb-8bbb-67c50aefcafa/image.jpg
773. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5a618c0b-ac35-454d-a922-0c5d8b7c4159/image.jpg
774. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ab47c9f-0184-4624-b82b-930763a001a2/image.jpg
775. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/7cf0482b-e62b-4f22-875f-3663f12e80d2/image.jpg
776. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/1575915f-ba85-40c7-8dc0-39db64cf4bbd/image.jpg
777. image.jpg

778. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
779. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
780. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
781. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
782. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
783. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
784. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
785. https://www.hostinger.com/in/tutorials/how-to-integrate-n8n-with-whatsapp
786. https://blog.n8n.io/whatsapp-bot/
787. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
788. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
789. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
790. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
791. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
792. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
793. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
794. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
795. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
796. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
797. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
798. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
799. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
800. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
801. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg

802. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg
803. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg
804. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3f51479c-fc4c-41fb-8bbb-67c50aefcafa/image.jpg
805. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5a618c0b-ac35-454d-a922-0c5d8b7c4159/image.jpg
806. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ab47c9f-0184-4624-b82b-930763a001a2/image.jpg
807. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/7cf0482b-e62b-4f22-875f-3663f12e80d2/image.jpg
808. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/1575915f-ba85-40c7-8dc0-39db64cf4bbd/image.jpg
809. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c933a2df-a2d3-44c6-8808-1af18c86708c/image.jpg
810. https://community.n8n.io/t/how-to-setup-whatsapp-trigger-node-for-self-hosted-n8n-using-ngrok/203679
811. https://github.com/enescingoz/awesome-n8n-templates/blob/main/WhatsApp/Building Your First WhatsApp Chatbot.json
812. https://www.youtube.com/watch?v=LekI91lSf_8&vl=en
813. https://green-api.com/en/docs/integration/n8n/echo-bot/
814. https://www.youtube.com/watch?v=A0OwvNOLNlw
815. https://www.youtube.com/watch?v=DU2hwUsMBBM
816. https://www.youtube.com/watch?v=MqnsLNGSbQc
817. https://n8n.io/integrations/whatsapp-trigger/
818. https://n8n.io/workflows/2465-building-your-first-whatsapp-chatbot/
819. https://community.n8n.io/t/problem-with-whatsapp-trigger-messages/192829
820. https://wassenger.com/blog/how-to-automate-whatsapp-replies-with-n8n-to-improve-customer-service-ready-to-use-workflow
821. https://community.n8n.io/t/whatsapp-trigger-only-for-messages-received/72310
822. https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-langchain.chattrigger/
823. https://www.youtube.com/watch?v=aJZVX6vEh4Q
824. https://community.n8n.io/t/how-to-configure-or-triggers-for-whatsapp-and-telegram/53040
825. https://nocodestartup.io/en/step-by-step-to-integrate-official-whatsapp-api-in-n8n/
826. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
827. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
828. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg

829. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
830. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
831. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
832. https://n8n.io/integrations/whatsapp-trigger/
833. https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-langchain.chattrigger/
834. https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.whatsapp/
835. https://www.hostinger.com/in/tutorials/how-to-integrate-n8n-with-whatsapp
836. https://n8n.io/workflows/2465-building-your-first-whatsapp-chatbot/
837. https://blog.n8n.io/whatsapp-bot/
838. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
839. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
840. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
841. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
842. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
843. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
844. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
845. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
846. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
847. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
848. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
849. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
850. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
851. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
852. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
853. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg

854. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
855. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
856. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
857. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg
858. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg
859. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3f51479c-fc4c-41fb-8bbb-67c50aefcafa/image.jpg
860. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5a618c0b-ac35-454d-a922-0c5d8b7c4159/image.jpg
861. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ab47c9f-0184-4624-b82b-930763a001a2/image.jpg
862. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/7cf0482b-e62b-4f22-875f-3663f12e80d2/image.jpg
863. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/1575915f-ba85-40c7-8dc0-39db64cf4bbd/image.jpg
864. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c933a2df-a2d3-44c6-8808-1af18c86708c/image.jpg
865. image.jpg
866. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
867. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
868. https://n8n.io/integrations/whatsapp-trigger/
869. https://community.n8n.io/t/whatsapp-trigger-only-for-messages-received/72310
870. https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-langchain.chattrigger/
871. https://n8n.io/workflows/2465-building-your-first-whatsapp-chatbot/
872. https://blog.n8n.io/whatsapp-bot/
873. image.jpg
874. image.jpg
875. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
876. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
877. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
878. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
879. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg

880. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
881. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
882. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
883. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
884. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
885. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
886. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
887. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
888. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
889. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
890. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
891. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
892. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
893. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
894. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
895. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg
896. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg
897. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3f51479c-fc4c-41fb-8bbb-67c50aefcafa/image.jpg
898. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5a618c0b-ac35-454d-a922-0c5d8b7c4159/image.jpg
899. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ab47c9f-0184-4624-b82b-930763a001a2/image.jpg
900. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/7cf0482b-e62b-4f22-875f-3663f12e80d2/image.jpg
901. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/1575915f-ba85-40c7-8dc0-39db64cf4bbd/image.jpg

902. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c933a2df-a2d3-44c6-8808-1af18c86708c/image.jpg
903. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/57f09e46-d598-4654-bb3c-a43d2e7333d2/image.jpg
904. image.jpg
905. image.jpg
906. image.jpg
907. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
908. https://n8n.io/integrations/whatsapp-trigger/
909. https://n8n.io/workflows/2465-building-your-first-whatsapp-chatbot/
910. https://blog.n8n.io/whatsapp-bot/
911. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
912. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
913. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
914. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
915. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
916. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
917. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
918. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
919. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
920. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
921. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
922. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
923. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
924. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
925. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
926. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg

927. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg
928. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg
929. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3f51479c-fc4c-41fb-8bbb-67c50aefcafa/image.jpg
930. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5a618c0b-ac35-454d-a922-0c5d8b7c4159/image.jpg
931. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ab47c9f-0184-4624-b82b-930763a001a2/image.jpg
932. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/7cf0482b-e62b-4f22-875f-3663f12e80d2/image.jpg
933. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/1575915f-ba85-40c7-8dc0-39db64cf4bbd/image.jpg
934. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c933a2df-a2d3-44c6-8808-1af18c86708c/image.jpg
935. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/57f09e46-d598-4654-bb3c-a43d2e7333d2/image.jpg
936. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d2e7f0cf-0bfa-40e2-9245-b9db3a30182c/image.jpg
937. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2bdbfe3f-8ef5-4d53-9293-10c46ce44b25/image.jpg
938. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9d2fb20f-9edf-40cf-a32b-f1b9d82431fe/image.jpg
939. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/785447dc-e259-42bc-87d3-9d2a6ec12f07/image.jpg
940. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/e85ff5e4-7d35-489b-b6eb-266c9a3e05ef/image.jpg
941. image.jpg
942. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
943. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
944. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
945. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg
946. https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger/
947. https://n8n.io/integrations/whatsapp-trigger/
948. https://blog.n8n.io/whatsapp-bot/
949. https://community.n8n.io/t/whatsapp-trigger-only-for-messages-received/72310
950. https://n8n.io/workflows/2465-building-your-first-whatsapp-chatbot/

951. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/51164a30-64a1-47e9-bb76-74fdc908daba/image.jpg
952. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9a3025fc-2542-497b-86d5-c48f901e9b72/image.jpg
953. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/449ad479-0c2d-4679-a529-dae3be4cb34f/image.jpg
954. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ee22233-8bfc-4c98-80bb-4809eab10a68/image.jpg
955. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2661483f-7b16-4bff-8623-b9b66ef10537/image.jpg
956. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3d066944-3ae5-423c-baf2-deb88319b889/image.jpg
957. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/37e4dcec-8d1a-4e02-b599-781921136cd8/image.jpg
958. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2107386e-4c4f-412f-b71a-2878c40bdadb/image.jpg
959. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/760329a4-fee2-46a9-a8eb-68f2f7d808ec/image.jpg
960. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/00dfb441-2f4f-4871-a321-3c7342bae121/image.jpg
961. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/575ff5d5-6aac-43ef-abd3-a131ed31f8eb/image.jpg
962. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2711bd87-3462-42b4-ac9e-1f6b46bd6535/image.jpg
963. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c177cf64-c948-4fd8-9edf-d10cffb46fd3/image.jpg
964. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/206b0ea3-fcb1-497a-96f9-c877ad37b205/image.jpg
965. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/ad2da1e9-8a80-4e2f-919b-fa3ca89b6c76/image.jpg
966. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/eb99b064-e73d-40ee-8469-45d95e75e8e5/image.jpg
967. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/80ce99d4-abae-420f-bbab-59ff7e02b542/image.jpg
968. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/0f37ae08-427c-44f5-91e0-b5df8c213401/image.jpg
969. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/3f51479c-fc4c-41fb-8bbb-67c50aefcafa/image.jpg
970. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5a618c0b-ac35-454d-a922-0c5d8b7c4159/image.jpg
971. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/5ab47c9f-0184-4624-b82b-930763a001a2/image.jpg
972. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/7cf0482b-e62b-4f22-875f-3663f12e80d2/image.jpg

973. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/1575915f-ba85-40c7-8dc0-39db64cf4bbd/image.jpg
974. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c933a2df-a2d3-44c6-8808-1af18c86708c/image.jpg
975. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/57f09e46-d598-4654-bb3c-a43d2e7333d2/image.jpg
976. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d2e7f0cf-0bfa-40e2-9245-b9db3a30182c/image.jpg
977. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/2bdbfe3f-8ef5-4d53-9293-10c46ce44b25/image.jpg
978. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/9d2fb20f-9edf-40cf-a32b-f1b9d82431fe/image.jpg
979. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/785447dc-e259-42bc-87d3-9d2a6ec12f07/image.jpg
980. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/e85ff5e4-7d35-489b-b6eb-266c9a3e05ef/image.jpg
981. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/db03bcf5-3fcf-4f6b-ba7e-0eaaef307220/image.jpg
982. https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-langchain.chattrigger/
983. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/81ea3603-09d6-475c-9471-a944de49e662/image.jpg
984. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/702e0b40-2460-4abd-b14b-2ebfe2053442/image.jpg
985. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/c8aae9d0-edea-44d2-9803-783ba8f8ff98/image.jpg
986. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/images/28763508/d9be5cdb-b395-4489-91b5-a2ef69b4dad0/image.jpg